

Mini Operations ERP — Software Requirements & Technical Specification

Version: 1.0 **Status:** Implementation-ready **Source of truth:** *Full-Stack Developer Technical Case Study — Mini Operations ERP*

Legend used throughout this document:

-  **REQUIREMENT** — stated explicitly (or logically mandated) by the case study. Not negotiable.
 -  **RECOMMENDATION** — an engineering decision made where the case study leaves the mechanism open. Justified inline. You may substitute an equivalent approach as long as the underlying requirement is still satisfied.
 -  **AMBIGUITY** — the case study does not specify this; flagged rather than silently resolved.
-

1. Executive Summary

The Mini Operations ERP is a small, production-shaped operations system covering one core flow:

Inventory → Work Order → Material Stock Check → Internal Transfer (or Shortage) → Customer Reservation

The system is a conventional three-tier web application: a React/TypeScript SPA, a Node.js/TypeScript REST API, and a PostgreSQL relational database. The two hardest technical problems in the system are **not** UI problems — they are (1) guaranteeing that concurrent stock reservations and transfers can never oversell or double-count inventory, and (2) keeping every state transition (work order, transfer) auditable and impossible to corrupt. This document is written so that correctness of those two problems is unambiguous and testable, and so that the "unannounced live-verification change" mentioned in the case study can be absorbed without a rewrite (see §35).

2. Project Scope

In scope ( directly from the case study):

- Auth with 3 roles (Admin, Operations User, Sales User), backend-enforced authorization.

- Inventory management (item, category, location, batch, physical/reserved/available quantity).
- Work orders with material stock check and automatic shortage calculation.
- Internal stock transfers between locations with a strict dispatch/receive lifecycle.
- Customer orders with concurrency-safe stock reservation.
- Relational database, backend validation, error handling, transactions, automated tests, environment-based configuration.
- 5 screens only: Login, Inventory, Work Orders, Internal Transfers, Customer Orders.

Explicitly out of scope (do not build): payroll, accounting/GL, HR, procurement/purchasing, supplier management, invoicing/billing, multi-tenant support, notifications/email, file uploads. If any of these appear tempting during implementation, they are scope creep relative to the case study and should be resisted.

 **Ambiguities called out up front** (resolved as  recommendations in the relevant sections, not silently assumed):

1. Whether Customer Orders support multiple line items or exactly one item per order — case study examples only ever show a single item. §14 resolves this.
2. Whether "Batch" is a first-class trackable entity (with batch number/expiry) or just a free-text attribute of inventory — §11 resolves this.
3. Whether transfers should support partial dispatch/receive today — the case study's *base* requirements imply full-quantity transfers only (partial receive is explicitly listed as a possible *future* live-verification change in §35), so partial transfer is **not** built now, but the schema is shaped to add it without migration surgery.
4. Whether Work Orders consume/deduct inventory on completion — the case study only asks for a **shortage calculation**, not automatic material deduction on completion. This document does **not** invent an auto-deduction rule; it is flagged as a candidate follow-up decision in §35.

3. Business Requirements

Restated as testable statements (each is traced in §37):

#	Requirement
BR-1	Users authenticate and are assigned exactly one of: Admin, Operations User, Sales User.
BR-2	Admin creates Work Orders.
BR-3	Operations User manages inventory and internal transfers.

#	Requirement
BR-4	Sales User creates customer orders and reserves stock.
BR-5	Inventory tracks Physical, Reserved, and Available quantity per item/location(/batch), where Available = Physical – Reserved.
BR-6	The system prevents negative inventory, invalid quantities, duplicate inventory transactions, and over-reservation.
BR-7	A Work Order can be checked against inventory at its location; a shortage is calculated automatically.
BR-8	If material is available elsewhere, an Internal Transfer can move stock between locations, following Requested → Dispatched → Received, with source decreasing only on dispatch and destination increasing only on receipt (never twice).
BR-9	A Sales User can reserve stock for a Customer Order; two concurrent reservations can never together exceed Available quantity.

4. Actors & Roles

4.1 Role → Permission → Resource Matrix

Resource / Action	Admin	Operations User	Sales User
Login / view own profile	✓	✓	✓
View inventory (read)	✓	✓	✓
Create/update inventory (stock levels, items, categories, locations)	✗	✓	✗

Resource / Action	Admin	Operations User	Sales User
Create Work Order	✓	✗	✗
View Work Orders	✓	✓	✓ (read-only)
Update Work Order status (Assigned → In Progress → Completed)	✓ (any)	✓ (if assigned to them)	✗
Run material stock check on a Work Order	✓	✓	✗
Create Internal Transfer (Request)	✗	✓	✗
Dispatch Internal Transfer	✗	✓	✗
Receive Internal Transfer	✗	✓	✗
View Internal Transfers	✓	✓	✓ (read-only)
Create Customer Order	✗	✗	✓
Reserve stock against a Customer Order	✗	✗	✓
View Customer Orders	✓	✓ (read-only)	✓
View users / manage roles	✓	✗	✗

● **RECOMMENDATION:** Admin gets full read access everywhere and can create Work Orders (per the case study). It does **not** automatically get write access to inventory or transfers — the case study explicitly assigns those to Operations User. Admin is not a superuser bypass; it is a distinct role with its own narrow write scope plus broad read scope. This keeps the permission

model honest and testable (Test 5 requires that *unauthorized* users are rejected — an over-privileged Admin would make that test meaningless for inventory/transfer endpoints).

⚠ **AMBIGUITY:** The case study doesn't say whether Admin can also do everything Operations/Sales can. Resolved conservatively above; easy to loosen later by editing the permission table + middleware config, not the code structure.

4.2 Authentication Flow

● **RECOMMENDATION** (case study does not mandate a specific mechanism):

1. **POST /auth/login** with email + password.
2. Server verifies password hash (bcrypt, cost factor 12), looks up role.
3. Server issues:
 - **Access token** — JWT, 15 minute expiry, signed with **JWT_SECRET**, contains { **sub: userId, role, iat, exp** }.
 - **Refresh token** — opaque random 256-bit token, 7 day expiry, stored hashed in a **refresh_tokens** table, delivered as an **httpOnly, Secure, SameSite=Strict** cookie (not accessible to JS — mitigates XSS token theft).
4. Access token is sent by the frontend in **Authorization: Bearer <token>** on every request; never stored in **localStorage** (mitigates XSS).
5. On 401 due to access-token expiry, the frontend calls **POST /auth/refresh** (refresh cookie sent automatically); server validates the refresh token against the stored hash, **rotates** it (issues a new refresh token, invalidates the old one — rotation detects token replay), and returns a new access token.
6. **POST /auth/logout** deletes the server-side refresh token record and clears the cookie.

sequenceDiagram

participant U as User

participant F as Frontend

participant A as API

participant D as Database

U->>F: enter credentials

F->>A: POST /auth/login

A->>D: lookup user by email

D-->>A: user + password_hash

A->>A: bcrypt.compare(password, hash)

A->>D: insert refresh_token (hashed)

A-->>F: access token (JSON) + refresh cookie (httpOnly)

F->>F: store access token in memory (React state)

F->>A: subsequent requests: Authorization: Bearer <token>

A-->>F: 401 (token expired)

F->>A: POST /auth/refresh (cookie auto-sent)

A->>D: validate + rotate refresh token

A-->>F: new access token + new refresh cookie

4.3 Authorization Flow / Guards

- **authenticate** middleware: verifies JWT signature + expiry, attaches `req.user = { id, role }`. Missing/invalid/expired token → **401 Unauthorized**.
- **authorize(...allowedRoles)** middleware: checks `req.user.role` against the route's allowed roles. Wrong role → **403 Forbidden**.
- Every mutating route (**POST**, **PATCH**, **DELETE**) on every resource **must** be wrapped in both middlewares — this is enforced by a lint rule / route-registration convention in §9, not left to per-controller discipline.  **REQUIREMENT** — "Backend authorization is mandatory," "Do not rely only on frontend permission hiding."
- **401 vs 403** — the distinction is enforced consistently:
 - 401 = "we don't know who you are" (no/expired/invalid token).
 - 403 = "we know who you are, and you're not allowed to do this."

4.4 Auth Attack Surface Mitigations

Attack	Mitigation
Brute-force login	Rate limit <code>/auth/login</code> (e.g. 10 attempts / 15 min / IP+email via <code>express-rate-limit</code>), generic error

Attack	Mitigation
	message ("invalid email or password") that doesn't reveal which field was wrong.
Token theft via XSS	Access token kept in memory only; refresh token in <code>httpOnly</code> cookie.
Refresh token replay	Rotation on every refresh; if an already-used (revoked) refresh token is presented, revoke the entire token family and force re-login.
CSRF on cookie-based refresh endpoint	<code>SameSite=Strict</code> cookie + <code>/auth/refresh</code> requires no other side effects beyond issuing tokens; state-changing endpoints all require the <code>Authorization</code> header (not just the cookie), so a forged cross-site request without JS-readable access token cannot act on the user's behalf.
Password storage compromise	bcrypt (cost 12) or argon2id; never log or return password hashes.
Session fixation / stale sessions	Access token short-lived (15 min); logout revokes refresh token server-side.

5. Functional Requirements

Grouped by module; each item is  unless marked otherwise.

Authentication & Roles

- Login with email/password; 3 fixed roles; JWT-based session; logout invalidates refresh token.

Inventory

- CRUD for Items, Categories, Locations (Operations User; Admin read-only elsewhere per §4.1).
- Inventory record per (Item, Location, Batch) storing Physical Quantity and Reserved Quantity; Available Quantity always derived correctly.

- Reject negative physical/reserved quantities, non-numeric/zero-or-below input where not allowed, duplicate transaction submissions, and reservations beyond available quantity.

Work Orders

- Admin creates a Work Order (Location, Item, Required Quantity, Assigned User, Status).
- Status lifecycle: Assigned → In Progress → Completed.
- Material stock check: compare Required Quantity to Available Quantity at the Work Order's Location; compute Shortage = $\max(0, \text{Required} - \text{Available})$.

Internal Transfers

- Create a transfer (Source Location, Destination Location, Item, Quantity) when material is available elsewhere.
- Lifecycle: Requested → Dispatched → Received.
- Dispatch reduces source inventory; destination is untouched until Receive; Receive increases destination inventory exactly once.

Customer Orders & Reservation

- Sales User creates a Customer Order and reserves inventory against it.
- Reservation increases Reserved Quantity (and therefore decreases Available) atomically; concurrent reservations can never jointly oversell.

Cross-cutting

- Backend validation on every write endpoint (never trust client-supplied totals/status).
- Structured error responses with stable error codes.
- Database transactions around every multi-row mutation (§20).
- Environment-based configuration (dev/test/prod), no hardcoded secrets.
- Automated tests, including the 5 mandatory scenarios (§27).

6. Non-Functional Requirements

Category	Requirement
Correctness	Inventory arithmetic must be exact (no floating point for quantities where the domain implies integers — see §17 validation rules);

Category	Requirement
	race conditions must not corrupt Reserved/Physical quantities.
Security	All mutating endpoints authenticated + role-checked server-side; passwords hashed; secrets in environment variables.
Portability	🔵 "Business logic should not depend heavily on one hosting provider or environment." No use of provider-specific services (no vendor-locked queues, storage, or serverless-only APIs); standard SQL via ORM; runs identically via Docker Compose locally or on any Node+Postgres host.
Maintainability	Layered backend (controller/service/repository), typed end-to-end (TypeScript + Prisma), single source of truth for state-machine rules.
Testability	Every business rule that matters for scoring (§ Evaluation criteria) has at least one automated test.
Observability	Structured logs, request IDs, health check endpoint (see §36).
Performance	Indexed foreign keys and lookup columns, paginated list endpoints, no N+1 queries (see §27).

7. UX / UI Specification

7.1 Design Philosophy

A serious, dense, desktop-first **operations dashboard** — not a marketing site. Optimized for people who use it 8 hours a day: high information density, predictable layout, minimal motion, fast keyboard-friendly forms. No illustrations, no decorative color, no large hero sections.

7.2 Layout System

Property	Value
Strategy	Desktop-first, responsive down to tablet ($\geq 768\text{px}$); mobile is "usable, not optimized" (● recommendation — case study emphasizes functionality over visual polish, and ERP back-office tools are overwhelmingly used on desktop).
Max content width	1440px, centered; sidebar fixed 240px, content area fluid.
Sidebar	Fixed left, 240px wide, collapsible to 64px (icons only), contains: logo, nav (Inventory, Work Orders, Transfers, Customer Orders — filtered by role), user menu + logout at bottom.
Topbar / header	56px height: page title + breadcrumb (left), search (center-left, page-specific), user avatar/role badge (right).
Page spacing	Content padding 24px; section vertical gap 24px; card internal padding 16px.
Grid system	12-column CSS grid, gutter 16px, used for form layouts and dashboard cards; tables ignore the grid and span full content width.
Column system	Forms: label above input, 2-column grid on desktop ($\geq 1024\text{px}$) collapsing to 1-column below.
Gap/spacing scale	4 / 8 / 12 / 16 / 24 / 32 / 48px (Tailwind default scale — <code>spacing-1</code> through <code>spacing-12</code>).

7.3 Color System

Token	Hex	Usage
Primary	#1D4ED8 (blue-700)	Primary buttons, active nav item, links, focus rings.
Primary hover	#1E40AF	Button/link hover state.
Secondary	#334155 (slate-700)	Secondary buttons, headings.
Background	#F8FAFC (slate-50)	App background.
Surface	#FFFFFF	Cards, tables, modals.
Border	#E2E8F0 (slate-200)	Dividers, table borders, input borders.
Text primary	#0F172A (slate-900)	Body text, headings.
Text secondary	#64748B (slate-500)	Helper text, table secondary column, timestamps.
Success	#15803D (green-700) / bg #DCFCE7	Completed, Received, success toasts.
Warning	#B45309 (amber-700) / bg #FEF3C7	In Progress, Dispatched, shortage warnings.
Error	#B91C1C (red-700) / bg #FEE2E2	Validation errors, insufficient stock, failed actions.
Info	#0369A1 (sky-700) / bg #E0F2FE	Neutral notices ("Requested" status, informational toasts).
Disabled	#94A3B8 text on #F1F5F9 bg	Disabled buttons/inputs.

7.4 Component-Level Design Rules

Element	Rule
Buttons	Primary (filled, blue) = the one main action per view; Secondary (outlined, slate) = alternate actions; Destructive (filled, red) =

Element	Rule
	cancel/reject-type actions, always behind a confirmation dialog.
Inputs	40px height, 1px border (Border token), 6px radius, red border + inline message on validation error.
Select/dropdown	Native-feel dropdown, same height/border as inputs; searchable when >20 options (e.g. Item picker).
Badges (status)	Pill shape, colored background per §7.3 mapping (see §23 state tables), never color-only — always paired with the text label.
Tables	Sticky header, zebra-free (border-only rows), right-aligned numeric columns, row-hover highlight, 44px row height.
Modals	Small (480px) for confirmations, Medium (640px) for single-entity forms (Create Work Order, Create Transfer). No modal is used for multi-step flows — those get dedicated drawers.
Toasts	Bottom-right, auto-dismiss 5s (errors: manual dismiss), one active toast at a time (queue additional).
Confirmation dialogs	Required before: Dispatch, Receive, status transitions that are irreversible, and any destructive action. Dialog states the consequence in plain language (e.g. "This will reduce inventory at Warehouse A by 40 units and cannot be undone.").
Loading skeletons	Gray animated blocks matching the shape of the table/card being loaded; never a blank screen or spinner-only for list views.

Element	Rule
Empty states	Icon + one-line message + primary action button where applicable (e.g. "No transfers yet — Create Transfer").
Error states	Inline banner at top of the affected panel with the server's error message (never a raw stack trace), plus a retry action where relevant.

8. Frontend Architecture

8.1 Goals

Type-safe, testable, small — five screens, no unnecessary abstraction layers, server is always the source of truth for business rules.

8.2 Stack (see §30 for full table)

React 18 + TypeScript, Vite, React Router, TanStack Query (server state), React Hook Form + Zod (forms/validation), Tailwind CSS, Axios (HTTP client).

8.3 Folder Structure

frontend/

```
|— src/
|
| |— api/           # thin API client functions per resource
|
| | |— client.ts    # axios instance + interceptors
|
| | |— auth.ts
|
| | |— inventory.ts
|
| | |— workOrders.ts
|
| | |— transfers.ts
|
| | |— orders.ts
```

- | |— components/
- | | |— layout/ # AppShell, Sidebar, Topbar, Breadcrumb
- | | |— ui/ # Button, Input, Select, Modal, Toast, Badge, Table...
- | | |— domain/ # StockAvailabilityIndicator, WorkOrderStatusBadge...
- | |— features/
- | | |— auth/ # LoginPage, useAuth hook
- | | |— inventory/ # InventoryPage + hooks + local components
- | | |— work-orders/
- | | |— transfers/
- | | |— customer-orders/
- | |— hooks/ # cross-feature hooks (usePagination, useDebounce)
- | |— lib/ # queryClient, zod schemas, formatters
- | |— routes/ # route definitions + ProtectedRoute/RoleGuard
- | |— types/ # shared TS types mirroring backend DTOs
- | |— App.tsx
- | |— main.tsx
- |— tests/
- |— .env.example
- |— vite.config.ts

8.4 Routing

Route	Component	Access
/login	LoginPage	Public
/inventory	InventoryPage	Admin (read), Operations (read/write), Sales (read)
/work-orders	WorkOrdersPage	Admin (read/write), Operations (read/write status), Sales (read)
/transfers	TransfersPage	Admin (read), Operations (read/write)
/orders	CustomerOrdersPage	Admin (read), Operations (read), Sales (read/write)
/	Redirect → first accessible route for the user's role	authenticated
*	404 page	any

● **RECOMMENDATION:** A `ProtectedRoute` wrapper checks `isAuthenticated` (redirects to `/login`, preserving intended destination). A `RoleGuard` wrapper checks role against the route's `allowedRoles` and renders a "403 — not authorized" in-app page (not a redirect) if the role doesn't match — this makes unauthorized access visible during manual QA rather than silently bouncing.

8.5 State Management

- **Server state** (inventory, work orders, transfers, orders): TanStack Query. Every list is a `useQuery`; every mutation is a `useMutation` that invalidates the relevant query keys on success (e.g. reserving stock invalidates both the order detail query and the inventory list query, since available quantity changed).
- **Client/UI state** (modal open/closed, selected filters, form state): local component state / React Hook Form. No global client-state library (Redux/Zustand) is needed at this scale — introducing one would be over-engineering for 5 screens.
- **Auth state:** a small `AuthContext` holding `{ user, accessToken, login(), logout() }`, backed by the in-memory token strategy from §4.2.

8.6 Forms & Validation

React Hook Form + Zod resolver. Every form's Zod schema is a client-side **mirror** of the backend's validation rules (§17) — duplicated intentionally for fast feedback, never treated as authoritative.

8.7 Error/Loading/Empty Handling (cross-cutting pattern)

Every data-fetching feature follows the same pattern: `isLoading` → skeleton; `isError` → inline error banner with retry; `data.length === 0` → empty state; otherwise → content. This is implemented once as a `<QueryBoundary>` wrapper component and reused on all 5 pages rather than repeated per page.

8.8 Accessibility & Performance

Semantic HTML, labeled form fields, focus-visible states using the Primary color, keyboard-operable modals (Esc to close, focus trap). Route-level code splitting (`React.lazy`) per feature; TanStack Query caching avoids redundant refetches; table virtualization is **not** needed at this data scale (● avoid over-engineering).

8.9 Environment Variables (frontend)

Variable	Purpose
<code>VITE_API_BASE_URL</code>	Backend API base URL.

9. Frontend Pages

Common conventions applied to all 5 pages: unauthorized users hitting a route via the guard see an in-app 403 message with a link back to their allowed home page; API errors render via the shared error banner pattern (§8.7); every table has server-side pagination, sorting, and (where applicable) filtering/search.

9.1 Login

Aspect	Spec
Route	<code>/login</code>
Access	Public (redirects away if already authenticated)

Aspect	Spec
Layout	Centered card (400px) on the Background color, no sidebar/topbar.
Fields	Email, Password.
Validation	Email format required; password required, min 1 char client-side (server enforces real policy at account-creation time, not at login).
Actions	"Log in" (primary button, disabled while submitting).
States	Loading: button spinner + disabled form. Error: inline banner "Invalid email or password." Success: redirect to role's default page.
API	POST /auth/login → { accessToken , user : { id , name , role } } (refresh token via cookie).
Business rules triggered	Rate limiting server-side (§4.4); no client-visible lockout message beyond the generic error (avoid user enumeration).

9.2 Inventory

Aspect	Spec
Route	/inventory
Access	All roles (read); create/update restricted to Operations User (buttons hidden and backend-enforced).
Layout	Page header ("Inventory") + primary action "Add Stock Entry" (Operations only) → table below.
Main sections	Filter bar (Location, Category, Item search), data table, "Add/Adjust Stock" modal.

Aspect	Spec
Grid	Full-width table, filter bar as a horizontal flex row above it.
Table columns	Item, Category, Location, Batch, Physical, Reserved, Available, Actions.
Filters	Location (select), Category (select), search by Item name/SKU.
Sorting	By Item name, Location, Available quantity.
Pagination	Server-side, 20 rows/page, page-number control.
Forms	"Add/Adjust Stock" modal: Item (searchable select), Location (select), Batch (optional text/select), Quantity delta, Reason (enum: Receipt/Adjustment). Operations User only.
Status indicators	StockAvailabilityIndicator : green if Available > 0, amber if Available is low relative to a configurable threshold (● recommendation, e.g. <10% of physical — cosmetic only, not a business rule), red if Available = 0.
Validation	Quantity must be a positive integer (or positive number if fractional units are enabled per item — see §17); required fields enforced before submit.
Loading/Empty/Error	Standard pattern (§8.7); empty state: "No inventory records yet."
Role-specific behavior	Sales/Admin see the table without any write controls.
API calls	GET <code>/inventory?location=&category=&search=&page=&sort=</code> , POST <code>/inventory</code> (create item-location-batch record) or PATCH <code>/inventory/:id</code> (quantity adjustment) — see §16 for the exact contract; GET

Aspect	Spec
	<code>/inventory/:id/transactions</code> for the audit drawer.
Business rules triggered	Negative-quantity and duplicate-transaction guards (§11); every adjustment writes an <code>inventory_transactions</code> row.
Navigation	Row click opens a read-only "Transaction History" drawer (<code>GET /inventory/:id/transactions</code>).

9.3 Work Orders

Aspect	Spec
Route	<code>/work-orders</code>
Access	Admin (create + full read), Operations User (read + status transitions + run stock check), Sales User (read-only).
Layout	Page header + "Create Work Order" button (Admin only) → filterable table.
Table columns	WO ID, Location, Item, Required Qty, Assigned User, Status, Created At, Actions.
Filters	Status, Location, Assigned User.
Sorting	Created date, Status.
Pagination	Server-side, 20/page.
Forms	"Create Work Order" modal (Admin): Location (select), Item (searchable select), Required Quantity (number), Assigned User (select, filtered to Operations Users).
Modals/drawers	"Stock Check" drawer: shows Required vs Available at the WO's location and the computed Shortage; if Shortage > 0, a "Create Transfer from this shortage" shortcut pre-fills the Transfer form (source = a location

Aspect	Spec
	with available stock, destination = this WO's location, item, quantity = shortage) —  recommendation, a UX convenience, not a new business rule.
Buttons/actions	"Run Stock Check", "Advance Status" (label changes: "Start" for Assigned→In Progress, "Complete" for In Progress→Completed), each behind a confirmation dialog.
Status indicators	<code>WorkOrderStatusComponent</code> badge: Assigned = Info (blue), In Progress = Warning (amber), Completed = Success (green).
Validation	Required Quantity > 0 integer; Assigned User must have the Operations User role (enforced server-side); status transition buttons disabled client-side if the transition isn't valid from the current status (server re-validates regardless).
Role-specific behavior	Only the assigned Operations User (or any Operations User — see §12 state machine table for the exact rule) can advance status; Sales sees no action buttons.
API calls	<code>GET /work-orders?...</code> , <code>POST /work-orders</code> (Admin), <code>PATCH /work-orders/:id/status</code> , <code>GET /work-orders/:id/stock-check</code> .
Business rules triggered	Shortage calculation (§12); status state machine (§22).
Navigation	Row click → detail drawer with stock-check + history.

9.4 Internal Transfers

Aspect	Spec
Route	<code>/transfers</code>

Aspect	Spec
Access	Admin, Sales User: read-only. Operations User: full read/write.
Layout	Page header + "Create Transfer" button (Operations only) → table.
Table columns	Transfer ID, Source, Destination, Item, Quantity, Status, Requested At, Actions.
Filters	Status, Source Location, Destination Location.
Forms	"Create Transfer" modal: Source Location (select), Destination Location (select, must differ from source — validated client + server), Item (select), Quantity (number). Shows live "Available at source" helper text fetched from inventory.
Buttons/actions	"Dispatch" (only when status = Requested), "Receive" (only when status = Dispatched) — both Operations-only, both behind a confirmation dialog stating the inventory effect.
Status indicators	TransferStatusComponent : Requested = Info, Dispatched = Warning, Received = Success.
Validation	Quantity ≤ available at source (checked client-side for UX, re-checked authoritatively server-side); Source ≠ Destination.
Role-specific behavior	Dispatch/Receive buttons are disabled/hidden for non-Operations roles and for transfers not in the correct status (e.g. "Receive" is not shown at all on a "Requested" transfer, preventing the double-receive UX path entirely — though the real guarantee is server-side, see §13/§22).

Aspect	Spec
Handle duplicate receive attempts	If a "Receive" click races with another and the server returns <code>409 CONFLICT / TRANSFER_ALREADY_RECEIVED</code> , the UI shows a toast ("This transfer was already received by someone else.") and refetches the row so the button disappears.
API calls	<code>GET /transfers?...</code> , <code>POST /transfers</code> , <code>PATCH /transfers/:id/dispatch</code> , <code>PATCH /transfers/:id/receive</code> .
Business rules triggered	Dispatch/receive inventory movement, duplicate-receive prevention (§13/§21).

9.5 Customer Orders

Aspect	Spec
Route	<code>/orders</code>
Access	Sales User: full read/write. Admin, Operations User: read-only.
Layout	Page header + "Create Order" button (Sales only) → table.
Table columns	Order ID, Customer, Item(s), Status, Created At, Created By, Actions.
Filters	Status, Sales User (Admin/Operations view).
Forms	"Create Order" modal: Customer Reference (text), then one or more line items: Item (select), Location (select), Quantity (number) — see §14 for the multi-item decision. Shows live "Available" for the selected item/location.
Buttons/actions	"Reserve" per line item (or "Reserve All" at the order level), disabled once already reserved.

Aspect	Spec
Validation	Quantity $\leq$ available at the moment of submit (server is authoritative — see concurrency conflict handling below).
Handle reservation conflict	If the server responds 409 CONFLICT / INSUFFICIENT_STOCK (because concurrent reservations consumed the stock between the user viewing the page and clicking Reserve), the UI shows an inline error on that line item: "Only X units are now available — please adjust the quantity," refetches the current available quantity, and does not partially apply the reservation.
API calls	GET /orders?... , POST /orders , POST /orders/:id/reserve .
Business rules triggered	Atomic reservation with row locking (§14/§21).

10. Component System

Component	Responsibility	Key Props	State	Events	Reused Globally?
AppShell	Sidebar + topbar + content outlet	children	none	—	Yes
Sidebar	Role-filtered navigation	role	active route (from router)	onNavigate	Yes
Topbar	Page title, breadcrumb, user menu	title , breadcrumb	menu open/closed	onLogout	Yes
ProtectedRoute	Redirect unauthenticated users	children	—	—	Yes

Component	Responsibility	Key Props	State	Events	Reused Globally?
RoleGuard	Render 403 if role mismatched	allowedRoles, children	—	—	Yes
DataTable	Generic sortable/paginated table shell	columns, rows, sort, page, onSortChange, onPageChange	none (controlled)	onRowClick	Yes
Pagination	Page controls	page, totalPages	none	onPageChange	Yes
SearchBar	Debounced text search input	value, onChange	internal debounce timer	onChange	Yes
FilterPanel	Group of filter selects	filters, values	none (controlled)	onChange	Yes
Select / MultiSelect	Styled dropdown	options, value	open/closed	onChange	Yes
FormField	Label + input + validation message wrapper	label, error, children	none	—	Yes
Validation Message	Inline field error text	message	none	—	Yes
Modal	Overlay dialog shell	open, onClose, size	focus trap state	onClose	Yes
ConfirmationDialog	Yes/No confirm with	message, onConfirm, onCancel	none	onConfirm, onCancel	Yes

Component	Responsibility	Key Props	State	Events	Reused Globally?
	consequence text				
Toast / ToastProvider	Success/error notifications	type, message	queue	auto-dismiss timer	Yes
Badge / StatusBadge	Colored status pill	variant	none	—	Yes
QuantityInput	Numeric input with min/step/integer enforcement	value, min, step	none	onChange	Yes
StockAvailabilityIndicator	Visual signal for available-qty health	available, physical	none	—	Domain (inventory)
WorkOrderStatusComponent	Status badge + valid-next-action	status	none	—	Domain (work orders)
TransferStatusComponent	Status badge + valid-next-action	status	none	—	Domain (transfers)
OrderStatusComponent	Status badge	status	none	—	Domain (orders)
LoadingSkeleton	Placeholder shape while loading	variant (table/card)	none	—	Yes

Component	Responsibility	Key Props	State	Events	Reused Globally?
EmptyState	Icon + message + optional CTA	message, action	none	—	Yes
ErrorState	Inline error banner + retry	message, onRetry	none	onRetry	Yes

11. Frontend Business Logic (Client vs Server)

Golden rule: the frontend may *pre-check* and *display* everything below for UX responsiveness, but the backend re-validates all of it authoritatively; the frontend never trusts its own pre-check as the final answer.

Area	Frontend does	Server is authoritative for
Inventory	Display Physical/Reserved/Available (as returned by API — never computed client-side from stale data); block obviously invalid input (negative, non-numeric, empty) before submit; show shortage visually.	The actual Available computation, all write validation, negative-inventory prevention, duplicate-transaction detection.
Work Orders	Collect Location/Item/Qty/Assignee; disable status-transition buttons that are invalid from the current known status; display the shortage number returned by <code>GET /work-orders/:id/stock-check</code> .	Shortage calculation itself, valid state transitions, role checks on who may transition.

Area	Frontend does	Server is authoritative for
Transfers	Collect Source/Destination/Item/Qty; show a live "available at source" hint; disable Dispatch/Receive buttons when status doesn't allow them; handle duplicate-receive error gracefully (retry-safe UI).	Whether quantity $\leq$ available at source, the dispatch/receive inventory mutation, preventing a transfer from being received twice under concurrent requests.
Customer Orders	Collect Item/Location/Qty; show live available stock; surface the exact conflict error text from the server when a reservation fails due to a race.	Atomic reservation, race-condition prevention, the true available quantity at commit time.

12. API Integration

12.1 Conventions

Aspect	Value
Style	REST over HTTPS, JSON bodies.
Base URL	<code>\${VITE_API_BASE_URL}/api/v1</code>
Auth header	<code>Authorization: Bearer <accessToken></code> on every authenticated request.
Success envelope	<code>{ "success": true, "data": ..., "meta"?: { "page", "pageSize", "total" } }</code>
Error envelope	See §24 (standard error format).
Pagination contract	Query params <code>page</code> (1-indexed), <code>pageSize</code> (default 20, max 100); response

Aspect	Value
	<code>meta.total</code> , <code>meta.page</code> , <code>meta.pageSize</code> .
Filtering contract	Plain query params per resource, e.g. <code>?status=DISPATCHED&location=3</code> .
Sorting contract	<code>?sort=field</code> or <code>?sort=-field</code> for descending.
Caching	TanStack Query cache only (client-side); no server-side response caching for mutable resources — inventory/order data must always be fresh (● recommendation: correctness over cache hit rate at this scale).
Invalidation	Every mutation's <code>onSuccess</code> invalidates the query keys it can affect — e.g. <code>reserve</code> invalidates <code>['orders', orderId]</code> and <code>['inventory']</code> ; <code>dispatch/receive</code> invalidate <code>['transfers']</code> and <code>['inventory']</code> .
Retry	TanStack Query default (3 retries with backoff) for <code>GET</code> only; mutations are never auto-retried (to avoid double-submitting a reservation/dispatch) — the client instead relies on idempotency keys where relevant (§21).
Request cancellation	TanStack Query auto-cancels superseded queries (e.g. fast filter changes) via <code>AbortController</code> .

12.2 API Integration Matrix

Page	User Action	Endpoint	Method	Auth	Role	Success	Error	UI Update
Login	Submit credentials	<code>/auth/login</code>	POST	No	Any	200 + tokens	401 invalid credentials, 429	Redirect to home

Page	User Action	Endpoint	Method	Auth	Role	Success	Error	UI Update
							rate-limited	
Inventory	Load list	/inventory	GET	Yes	Any	200 + rows	401/403	Render table
Inventory	Adjust stock	/inventory/:id	PATCH	Yes	Operations	200 + updated row	400 invalid qty, 409 duplicate txn, 403	Update row, toast, invalidate list
Work Orders	Create	/work-orders	POST	Yes	Admin	201 + WO	400, 403	Add row, toast, close modal
Work Orders	Run stock check	/work-orders/:id/stock-check	GET	Yes	Admin, Operations	200 + {required, available, shortage}	403, 404	Render drawer
Work Orders	Advance status	/work-orders/:id/status	PATCH	Yes	Operations (assigned)	200 + WO	409 invalid transition, 403	Update badge, toast
Transfers	Create	/transfers	POST	Yes	Operations	201 + Transfer	400 insufficient stock at source, 403	Add row
Transfers	Dispatch	/transfers/:	PATCH	Yes	Operations	200 + Transfer	409 invalid	Update badge,

Page	User Action	Endpoint	Method	Auth	Role	Success	Error	UI Update
		id/dispatch				, inventory updated	status, 400 insufficient stock, 403	invalidate inventory
Transfers	Receive	/transfers/:id/receive	PATCH	Yes	Operations	200 + Transfer, inventory updated	409 already received / wrong status, 403	Update badge, invalidate inventory
Orders	Create	/orders	POST	Yes	Sales	201 + Order	400, 403	Add row
Orders	Reserve	/orders/:id/reserve	POST	Yes	Sales	200 + Reservation	409 INSUFFICIENT_STOCK (concurrent conflict), 400, 403	Update order + inventory, or show conflict inline

13. Backend Architecture

13.1 Layered Architecture

HTTP Request

→ Route (method + path definition, wires middleware)

→ authenticate middleware (JWT)

- authorize(role) middleware
- validate middleware (Zod schema on body/params/query)
- Controller (thin: parse req → call service → shape response)
- Service (business rules, orchestrates transactions)
- Repository (Prisma client calls, one per entity)
- PostgreSQL

Controllers contain **no** business logic — only request/response shaping. All business rules (shortage calculation, state machines, locking) live in Services, which is what unit tests target directly (in addition to API-level integration tests).

13.2 Folder Structure

backend/

```
|— src/
|  |— config/          # env loading + validation (zod), constants
|  |— middleware/
|  |  |— authenticate.ts
|  |  |— authorize.ts
|  |  |— validate.ts
|  |  |— errorHandler.ts
|  |— modules/
|  |  |— auth/         # controller, service, routes, schemas
|  |  |— users/
|  |  |— locations/
|  |  |— categories/
```

```
| | └─ items/
| | └─ inventory/
| | └─ workOrders/
| | └─ transfers/
| | └─ orders/
| |   └─ orders.controller.ts
| |   └─ orders.service.ts
| |   └─ orders.repository.ts
| |   └─ orders.routes.ts
| |   └─ orders.schemas.ts
| └─ db/
| | └─ prisma.ts      # singleton Prisma client
| | └─ migrations/
| └─ lib/             # logger, errors, jwt helpers
| └─ app.ts           # express app assembly
|   └─ server.ts      # entrypoint
└─ prisma/
  └─ schema.prisma
  └─ seed.ts
└─ tests/
  └─ unit/
```


| |— integration/
| |— concurrency/
|— .env.example
|— openapi.yaml

13.3 Cross-Cutting Concerns

- **Logging:** structured JSON logs (pino), one line per request with request ID, method, path, status, duration, `userId`.
- **Config:** all env vars validated at boot via a Zod schema in `config/index.ts` — the app refuses to start if a required var is missing (fail fast, not fail silently in production).
- **API docs:** OpenAPI 3.0 spec (`openapi.yaml`), served via Swagger UI at `/docs` in non-production, matching §16 exactly.
- **Migrations:** Prisma Migrate, one migration file per schema change, checked into git (supports the "reasonable git history" submission requirement).
- **Seed data:** `prisma/seed.ts` creates one user per role, 2 locations, a handful of items/categories, and baseline inventory — enough to demo the full flow end-to-end.

14. Database Architecture

● **REQUIREMENT:** relational database. **Chosen: PostgreSQL** (see §30 for rationale) via Prisma ORM.

14.1 Available Quantity: Stored, Calculated, or Derived View?

Three options were evaluated:

Option	Pros	Cons
A. Store <code>available_quantity</code> as an independently-written column	Fast reads	Update-anomaly risk: every reservation/transfer write must remember to update three columns in sync; a missed update silently corrupts the invariant. Rejected.

Option	Pros	Cons
B. Compute in application code on every read, never persisted	No sync risk	Every consumer of inventory data (reports, other queries, ad-hoc SQL) must remember the formula; easy to get wrong outside the app layer; can't be indexed or filtered on efficiently in SQL. Rejected as sole mechanism.
C. Generated column at the database level (<code>available_quantity = physical_quantity - reserved_quantity, GENERATED ALWAYS AS (...) STORED</code>)	Single source of truth enforced by the database itself — impossible for any writer (app code, migration script, manual SQL) to make it inconsistent; still indexable/filterable/sortable like a normal column	Postgres recomputes it on every row write (negligible cost at this scale); cannot be directly written to (correct — it shouldn't be). Chosen.

● **RECOMMENDATION:** Option C. `physical_quantity` and `reserved_quantity` are the only columns anything ever writes to; `available_quantity` is a Postgres `GENERATED ALWAYS AS (physical_quantity - reserved_quantity) STORED` column with a `CHECK (available_quantity >= 0)` constraint alongside it. This makes over-reservation structurally impossible to persist, not just application-logic-dependent — the database itself is the last line of defense (see §21/§26).

14.2 Relationship Overview

- `items` **1—N** `inventory` (an item exists at many location/batch combinations)
- `locations` **1—N** `inventory`
- `batches` **1—N** `inventory` (optional grouping; see §11.1 below)
- `categories` **1—N** `items`
- `inventory` **1—N** `inventory_transactions` (audit trail)
- `users` **N—1** `roles` (each user has exactly one role)
- `work_orders` **N—1** `locations`, **N—1** `items`, **N—1** `users` (assignee), **N—1** `users` (creator)
- `internal_transfers` **N—1** `locations` (×2: source, destination), **N—1** `items`
- `customer_orders` **1—N** `customer_order_items`
- `customer_order_items` **N—1** `items`, **N—1** `locations`, **1—1** `reservations` (a reservation belongs to exactly one order line; see §14 business rule)

No many-to-many relationships are required at this scope.

15. Database Schema

15.1 roles

Column	Type	Constraints
id	smallint	PK
name	varchar(30)	UNIQUE, NOT NULL — ADMIN, OPERATIONS, SALES

● A lookup table rather than a hardcoded enum on `users`, so a 4th role can be added later without a schema migration touching the `users` table shape — directly supports §35 extensibility.

15.2 users

Column	Type	Constraints
id	uuid	PK, default <code>gen_random_uuid()</code>
name	varchar(100)	NOT NULL
email	varchar(255)	UNIQUE, NOT NULL
password_hash	varchar(255)	NOT NULL
role_id	smallint	FK → roles.id, NOT NULL, ON DELETE RESTRICT
location_id	uuid	FK → locations.id, NULLABLE, ON DELETE SET NULL
is_active	boolean	NOT NULL, default <code>true</code>
created_at	timestampz	NOT NULL, default <code>now()</code>
updated_at	timestampz	NOT NULL, default <code>now()</code>

Index: `users(email)` (unique index doubles as lookup index).

● **RECOMMENDATION:** `location_id` is nullable and **unused by any authorization logic today**. It exists purely as an extensibility hook for the case study's explicitly-anticipated live-verification "Change 4: restrict users to their assigned location" (§35) — adding the enforcement later is a middleware change, not a schema migration.

15.3 `refresh_tokens`

Column	Type	Constraints
<code>id</code>	<code>uuid</code>	PK
<code>user_id</code>	<code>uuid</code>	FK → <code>users.id</code> , NOT NULL, ON DELETE CASCADE
<code>token_hash</code>	<code>varchar(255)</code>	NOT NULL
<code>family_id</code>	<code>uuid</code>	NOT NULL (groups a rotation chain; enables "revoke whole family on reuse")
<code>expires_at</code>	<code>timestampz</code>	NOT NULL
<code>revoked_at</code>	<code>timestampz</code>	NULLABLE
<code>created_at</code>	<code>timestampz</code>	NOT NULL, default <code>now()</code>

Index: `refresh_tokens(user_id)`, `refresh_tokens(family_id)`.

15.4 `locations`

Column	Type	Constraints
<code>id</code>	<code>uuid</code>	PK
<code>code</code>	<code>varchar(20)</code>	UNIQUE, NOT NULL
<code>name</code>	<code>varchar(100)</code>	NOT NULL
<code>is_active</code>	<code>boolean</code>	NOT NULL, default <code>true</code>
<code>created_at</code>	<code>timestampz</code>	NOT NULL, default <code>now()</code>

15.5 categories

Column	Type	Constraints
id	uuid	PK
name	varchar(100)	UNIQUE, NOT NULL
created_at	timestampz	NOT NULL, default <code>now()</code>

15.6 items

Column	Type	Constraints
id	uuid	PK
sku	varchar(50)	UNIQUE, NOT NULL
name	varchar(150)	NOT NULL
category_id	uuid	FK → categories.id, NOT NULL, ON DELETE RESTRICT
unit_of_measure	varchar(20)	NOT NULL, default <code>'EA'</code>
allow_fractional	boolean	NOT NULL, default <code>false</code> —  drives whether quantities for this item may be decimal (see §17)
is_active	boolean	NOT NULL, default <code>true</code>
created_at	timestampz	NOT NULL, default <code>now()</code>

Index: `items(category_id)`, `items(sku)`.

15.7 batches (ambiguity resolution)

The case study lists "Batch" as an Inventory field but gives no example of batch semantics (no expiry, no lot number shown).  **RECOMMENDATION:** model `batches` as a first-class but minimal entity (batch number only, optional expiry) rather than a free-text string on `inventory`, because (a) it keeps `inventory`'s uniqueness constraint clean, and (b) it's a natural, low-cost extensibility point if lot/expiry tracking is asked for later, without inventing unused columns now.

Column	Type	Constraints
id	uuid	PK
item_id	uuid	FK → items.id, NOT NULL, ON DELETE CASCADE
batch_number	varchar(50)	NOT NULL
expiry_date	date	NULLABLE
created_at	timestampz	NOT NULL, default <code>now()</code>

Constraint: `UNIQUE(item_id, batch_number)`.

● For items that don't need batch tracking, a single sentinel batch row per item, e.g. `batch_number = 'DEFAULT'`, is created automatically when the item is created. This keeps `inventory`'s `batch_id` **NOT NULL**, which in turn keeps its uniqueness constraint simple (see below) instead of fighting Postgres's "NULLs are distinct" behavior in a `UNIQUE` index.

15.8 `inventory`

Column	Type	Constraints
id	uuid	PK
item_id	uuid	FK → items.id, NOT NULL, ON DELETE RESTRICT
location_id	uuid	FK → locations.id, NOT NULL, ON DELETE RESTRICT
batch_id	uuid	FK → batches.id, NOT NULL, ON DELETE RESTRICT
physical_quantity	numeric(14,3)	NOT NULL, default 0, <code>CHECK (physical_quantity >= 0)</code>
reserved_quantity	numeric(14,3)	NOT NULL, default 0, <code>CHECK (reserved_quantity >= 0)</code>

Column	Type	Constraints
available_quantity	numeric(14,3)	GENERATED ALWAYS AS (physical_quantity - reserved_quantity) STORED, CHECK (available_quantity >= 0)
version	integer	NOT NULL, default 0 — optimistic-lock helper, see §21
updated_at	timestampz	NOT NULL, default now()

Constraint: `UNIQUE(item_id, location_id, batch_id)`. Indexes: `inventory(item_id, location_id)`, `inventory(location_id)`.

● The `CHECK (reserved_quantity <= physical_quantity)` invariant is implied by the `available_quantity >= 0` check (since `available = physical - reserved`), so a separate redundant constraint isn't needed — one check enforces both.

15.9 `inventory_transactions` (audit log — §26)

Column	Type	Constraints
id	uuid	PK
inventory_id	uuid	FK → inventory.id, NOT NULL, ON DELETE RESTRICT
transaction_type	varchar(30)	NOT NULL — RECEIPT, ADJUSTMENT, TRANSFER_OUT, TRANSFER_IN, RESERVATION, RESERVATION_RELEASE
quantity_change	numeric(14,3)	NOT NULL — signed
physical_before	numeric(14,3)	NOT NULL
physical_after	numeric(14,3)	NOT NULL

Column	Type	Constraints
reserved_before	numeric(14,3)	NOT NULL
reserved_after	numeric(14,3)	NOT NULL
reference_type	varchar(30)	NULLABLE — WORK_ORDER, TRANSFER, ORDER, MANUAL
reference_id	uuid	NULLABLE
idempotency_key	varchar(100)	NULLABLE, UNIQUE — see §21 duplicate-transaction prevention
performed_by	uuid	FK → users.id, NOT NULL, ON DELETE RESTRICT
created_at	timestampz	NOT NULL, default now()

Indexes: `inventory_transactions(inventory_id, created_at)`, `inventory_transactions(reference_type, reference_id)`, unique index on `idempotency_key` where not null.

15.10 work_orders

Column	Type	Constraints
id	uuid	PK
location_id	uuid	FK → locations.id, NOT NULL, ON DELETE RESTRICT
item_id	uuid	FK → items.id, NOT NULL, ON DELETE RESTRICT
required_quantity	numeric(14,3)	NOT NULL, CHECK (required_quantity > 0)
assigned_user_id	uuid	FK → users.id, NOT NULL, ON DELETE RESTRICT

Column	Type	Constraints
created_by	uuid	FK → users.id, NOT NULL, ON DELETE RESTRICT
status	varchar(20)	NOT NULL, default 'ASSIGNED', CHECK (status IN ('ASSIGNED', 'IN_PROGRESS', 'COMPLETED'))
created_at	timestampz	NOT NULL, default now()
updated_at	timestampz	NOT NULL, default now()

Indexes: `work_orders(status)`, `work_orders(location_id)`, `work_orders(assigned_user_id)`.

15.11 internal_transfers

Column	Type	Constraints
id	uuid	PK
source_location_id	uuid	FK → locations.id, NOT NULL, ON DELETE RESTRICT
destination_location_id	uuid	FK → locations.id, NOT NULL, ON DELETE RESTRICT, CHECK (source_location_id <> destination_location_id)
item_id	uuid	FK → items.id, NOT NULL, ON DELETE RESTRICT
batch_id	uuid	FK → batches.id, NOT NULL, ON DELETE RESTRICT
quantity	numeric(14,3)	NOT NULL, CHECK (quantity > 0)

Column	Type	Constraints
status	varchar(20)	NOT NULL, default 'REQUESTED', CHECK (status IN ('REQUESTED', 'DISPATCHED', 'RECEIVED'))
requested_by	uuid	FK → users.id, NOT NULL
dispatched_by	uuid	FK → users.id, NULLABLE
dispatched_at	timestampz	NULLABLE
received_by	uuid	FK → users.id, NULLABLE
received_at	timestampz	NULLABLE
created_at	timestampz	NOT NULL, default now()

Indexes: `internal_transfers(status)`,
`internal_transfers(source_location_id)`,
`internal_transfers(destination_location_id)`.

● `received_at IS NOT NULL` is the structural guard the service layer checks (in a locked row read) before allowing a second receive — see §13/§21 for the exact mechanism; the `status` check constraint is the second, independent line of defense.

15.12 customer_orders

Column	Type	Constraints
id	uuid	PK
customer_reference	varchar(150)	NOT NULL
created_by	uuid	FK → users.id, NOT NULL, ON DELETE RESTRICT
status	varchar(20)	NOT NULL, default 'PENDING', CHECK (status IN ('PENDING', 'PARTIALLY_

Column	Type	Constraints
		RESERVED', 'RESERVED', 'CANCELLED'))
created_at	timestamptz	NOT NULL, default now()

⚠️ AMBIGUITY / 🟢 RECOMMENDATION: the case study's examples show one item per order, but a real "Customer Order" concept in an ERP is naturally multi-line. Modeling a `customer_order_items` child table costs nothing extra structurally and matches how the domain actually works, while still trivially supporting the single-item case (an order with one line). It also cleanly supports the anticipated "cancel an order and release its inventory" future change (§35) at the line level. This is called out explicitly rather than silently assumed.

15.13 `customer_order_items`

Column	Type	Constraints
id	uuid	PK
customer_order_id	uuid	FK → customer_orders.id, NOT NULL, ON DELETE CASCADE
item_id	uuid	FK → items.id, NOT NULL, ON DELETE RESTRICT
location_id	uuid	FK → locations.id, NOT NULL, ON DELETE RESTRICT
batch_id	uuid	FK → batches.id, NOT NULL, ON DELETE RESTRICT
quantity_requested	numeric(14,3)	NOT NULL, CHECK (quantity_requested > 0)
status	varchar(20)	NOT NULL, default 'PENDING', CHECK (status IN ('PENDING', 'RESERVED', 'CANCELLED'))

Column	Type	Constraints
created_at	timestampz	NOT NULL, default <code>now()</code>

15.14 reservations

Column	Type	Constraints
id	uuid	PK
customer_order_item_id	uuid	FK → customer_order_items.id, NOT NULL, UNIQUE, ON DELETE RESTRICT
inventory_id	uuid	FK → inventory.id, NOT NULL, ON DELETE RESTRICT
quantity	numeric(14,3)	NOT NULL, CHECK (quantity > 0)
status	varchar(20)	NOT NULL, default 'ACTIVE', CHECK (status IN ('ACTIVE', 'RELEASED', 'FULFILLED'))
created_at	timestampz	NOT NULL, default <code>now()</code>

`UNIQUE(customer_order_item_id)` enforces a 1:1 between an order line and its reservation — an order line is reserved exactly once (re-reservation after a release, if that future feature is added, would create a new row, keeping history intact).

15.15 Entity-Relationship Diagram

erDiagram

ROLES ||--o{ USERS : has

LOCATIONS ||--o{ USERS : "assigned to (nullable)"

USERS ||--o{ REFRESH_TOKENS : owns

CATEGORIES ||--o{ ITEMS : groups

ITEMS ||--o{ BATCHES : has

ITEMS ||--o{ INVENTORY : "stocked as"

LOCATIONS ||--o{ INVENTORY : holds

BATCHES ||--o{ INVENTORY : "grouped by"

INVENTORY ||--o{ INVENTORY_TRANSACTIONS : logs

USERS ||--o{ INVENTORY_TRANSACTIONS : performs

LOCATIONS ||--o{ WORK_ORDERS : "located at"

ITEMS ||--o{ WORK_ORDERS : requires

USERS ||--o{ WORK_ORDERS : "assigned / created"

LOCATIONS ||--o{ INTERNAL_TRANSFERS : "source / destination"

ITEMS ||--o{ INTERNAL_TRANSFERS : moves

USERS ||--o{ INTERNAL_TRANSFERS : "requests / dispatches / receives"

CUSTOMER_ORDERS ||--o{ CUSTOMER_ORDER_ITEMS : contains

ITEMS ||--o{ CUSTOMER_ORDER_ITEMS : references

LOCATIONS ||--o{ CUSTOMER_ORDER_ITEMS : "fulfilled from"

USERS ||--o{ CUSTOMER_ORDERS : creates

CUSTOMER_ORDER_ITEMS ||--o{ RESERVATIONS : reserved_as

INVENTORY ||--o{ RESERVATIONS : "reserved against"

16. Business Logic / Services

Service	Responsibilities	Transaction Boundary	Authorization
AuthenticationService	Verify credentials, issue/rotate/revoke tokens	Single-row read + single-row insert (refresh token) — no multi-table transaction needed	Public (login), authenticated (logout/refresh)
UserService	Lookup users, list Operations Users for WO assignment	Read-only	Admin for listing all users
LocationService, ItemService	CRUD for reference data	Single-row writes	Operations (write), any (read)
InventoryService	Read inventory, apply stock adjustments (receipts/manual corrections), enforce non-negative/duplicate-transaction rules	Wraps adjustment + <code>inventory_transactions</code> insert in one DB transaction	Operations (write), any (read)
InventoryTransactionService	Query audit history per inventory record	Read-only	any (read)
StockCheckService	Compare a Work Order's required quantity to available quantity at its location; compute shortage	Read-only	Admin, Operations
WorkOrderService	Create WO (Admin), validate assignee is an Operations User, enforce status state machine	Create = single insert; status update = single-row update guarded by current-status check	Admin (create), Operations (status transitions)
TransferService	Create transfer (Requested),	Dispatch and Receive are each a	Operations only

Service	Responsibilities	Transaction Boundary	Authorization
	Dispatch (decrement source + insert transaction), Receive (increment destination + insert transaction), prevent double-receive	single DB transaction: row-lock inventory + transfer, validate, mutate, log	
ReservationService	Atomically reserve stock against an order line with row-level locking; prevent oversell	Single DB transaction per reservation: lock inventory row, validate available $\geq$ requested, increment reserved_quantity, insert reservation + transaction log	Sales only
CustomerOrderService	Create order + line items; orchestrate per-line reservation calls	Order creation = single transaction (header + lines); reservation delegated to ReservationService per line	Sales (write), any (read)

Failure scenarios handled by every service uniformly: invalid input → 400

VALIDATION_ERROR; referenced entity missing → 404 NOT_FOUND; role mismatch → 403

FORBIDDEN; business-rule violation (insufficient stock, invalid transition, duplicate) → 409

CONFLICT with a specific error.code; unexpected DB/driver error → 500 INTERNAL_ERROR (logged with full detail server-side, generic message returned to the client).

17. API Specification

All endpoints are versioned under /api/v1. All (except /auth/login) require Authorization: Bearer <token>.

17.1 Authentication

POST /auth/login

- Auth: none. Body: { "email": string, "password": string }.
- 200: { success: true, data: { accessToken, user: { id, name, role } } } (+ refresh cookie).
- Errors: 400 malformed body, 401 INVALID_CREDENTIALS, 429 RATE_LIMITED.

POST /auth/refresh — Auth: refresh cookie. 200: new { accessToken } + rotated cookie. Errors: 401 INVALID_REFRESH_TOKEN.

POST /auth/logout — Auth: required. 204 No Content. Revokes refresh token.

GET /auth/me — Auth: required. 200: { id, name, email, role }.

17.2 Inventory

GET /inventory — Role: any. Query: location, category, item, search, page, pageSize, sort. 200: { data: [{ id, item: {id,name,sku}, location: {id,name}, batch: {id,batchNumber}, physicalQuantity, reservedQuantity, availableQuantity }], meta }.

GET /inventory/:id — Role: any. 200: single record as above. 404 NOT_FOUND.

POST /inventory — Role: Operations. Body: { itemId, locationId, batchId?, initialQuantity }. Creates a new item/location/batch inventory row (defaults batchId to the item's DEFAULT batch if omitted). 201. Errors: 400 VALIDATION_ERROR, 409 DUPLICATE_INVENTORY_RECORD (unique constraint hit).

PATCH /inventory/:id — Role: Operations. Body: { quantityChange: number, reason: 'RECEIPT'|'ADJUSTMENT', idempotencyKey?: string }. 200: updated record. Errors: 400 VALIDATION_ERROR (e.g. would drive physical below 0), 409 DUPLICATE_TRANSACTION (idempotency key already used), 403.

GET /inventory/:id/transactions — Role: any. Query: page, pageSize. 200: paginated inventory_transactions rows.

17.3 Work Orders

GET /work-orders — Role: any. Query: status, location, assignedUser, page, sort.

POST /work-orders — Role: Admin. Body: { `locationId`, `itemId`, `requiredQuantity`, `assignedUserId` }. Validates `assignedUserId` has role `OPERATIONS`. 201. Errors: 400, 403, 404 (location/item/user not found).

GET /work-orders/:id/stock-check — Role: Admin, Operations. 200: { `requiredQuantity`, `availableQuantity`, `shortage` }.

PATCH /work-orders/:id/status — Role: Operations (must be the assigned user — see §22 for the exact rule). Body: { `status`: 'IN_PROGRESS' | 'COMPLETED' }. 200: updated WO. Errors: 409 `INVALID_STATUS_TRANSITION`, 403 `NOT_ASSIGNED_TO_WORK_ORDER`.

17.4 Internal Transfers

GET /transfers — Role: any. Query: `status`, `sourceLocation`, `destinationLocation`, `page`, `sort`.

POST /transfers — Role: Operations. Body: { `sourceLocationId`, `destinationLocationId`, `itemId`, `batchId?`, `quantity` }. Validates `sourceLocationId` $\neq$ `destinationLocationId` and `quantity` $\leq$ available at source. 201, status `REQUESTED`. Errors: 400 `INSUFFICIENT_STOCK`, 400 `SAME_SOURCE_AND_DESTINATION`.

PATCH /transfers/:id/dispatch — Role: Operations. No body required. 200: updated transfer (status `DISPATCHED`), source inventory decremented. Errors: 409 `INVALID_STATUS_TRANSITION` (not in `REQUESTED`), 400 `INSUFFICIENT_STOCK` (stock changed since request).

PATCH /transfers/:id/receive — Role: Operations. No body required. 200: updated transfer (status `RECEIVED`), destination inventory incremented. Errors: 409 `TRANSFER_ALREADY_RECEIVED` / `INVALID_STATUS_TRANSITION` (not in `DISPATCHED`).

17.5 Customer Orders

GET /orders — Role: any. Query: `status`, `createdBy`, `page`, `sort`.

POST /orders — Role: Sales. Body: { `customerReference`, `items`: [{ `itemId`, `locationId`, `batchId?`, `quantity` }] }. 201: order + line items, each line `status` = `PENDING`.

POST /orders/:id/reserve — Role: Sales. Body: { `orderItemId` } (reserve one line —  keeps the concurrency-critical operation scoped to a single inventory row per call; reserving "the whole order" in the frontend is implemented as N sequential calls to this endpoint, one per line, so a partial failure on one line doesn't roll back an already-successful line). 200: {

`reservation, updatedInventory }`. Errors: 409 `INSUFFICIENT_STOCK` (see §14/§21 — this is the concurrency-conflict response), 409 `ALREADY_RESERVED`, 403.

17.6 Sample JSON — Reservation Conflict

```
{  
  
  "success": false,  
  
  "error": {  
  
    "code": "INSUFFICIENT_STOCK",  
  
    "message": "Only 20 units are available for this item at the requested location.",  
  
    "details": {  
  
      "requested": 50,  
  
      "available": 20,  
  
      "inventoryId": "b3a1e2b0-..."  
  
    }  
  
  }  
  
}
```

18. Validation

18.1 Why Server-Side Validation Is Authoritative

The client cannot be trusted: requests can be replayed, scripted, or sent by a modified client. Every rule that protects a business invariant (stock levels, valid transitions, authorization) must be re-checked on the server inside the same transaction that performs the mutation — client-side validation exists purely to give the user fast feedback, and is explicitly documented as non-authoritative everywhere in this spec.

18.2 Validation Matrix

Field	Client Validation	Server Validation	DB Constraint
email	format, required	format, required, existence check on login	UNIQUE
password	required, min length	required, verified against bcrypt hash	—
quantity (any)	required, numeric, > 0, integer unless item allows fractional	same, plus re-checked against current DB state at write time	CHECK (> 0) on relevant tables
itemId / locationId / batchId / userId	required, must be a valid selected option (from a fetched list)	required, existence check (404 if not found), active-record check (<code>is_active = true</code>)	FK constraint
work order <code>assignedUserId</code>	must be selected from an Operations-User-filtered list	must actually have role OPERATIONS	none (business rule, not a DB-level polymorphic constraint)
transfer <code>sourceLocationId</code> / <code>destinationLocationId</code>	must differ (inline error if equal)	must differ	CHECK (source <> destination)
transfer/reservation quantity vs available	live "available" hint shown, submit blocked if visibly exceeding	re-fetches available inside the transaction (not from the earlier hint) before allowing the mutation	CHECK (<code>available_quantity >= 0</code>) — final backstop
work order status transition	buttons disabled for invalid transitions	current status re-read inside the transaction; transition	CHECK (status IN (...))

Field	Client Validation	Server Validation	DB Constraint
	given last-known status	validated against the state machine table (§22)	
transfer status transition	Dispatch/Receive buttons only shown for the correct current status	same pattern as above	<code>CHECK (status IN (...))</code>
idempotency key (stock adjustment)	generated client-side per submit attempt (UUID), resent unchanged on retry	unique index lookup; if seen before, returns the original result instead of double-applying	<code>UNIQUE</code> on <code>idempotency_key</code>
duplicate receive	UI hides the Receive button once status flips	conditional update (<code>WHERE status = 'DISPATCHED'</code>) — 0 rows affected ⇒ 409	<code>CHECK (status IN (...))</code> + application guard

19. Authentication (Consolidated)

See §4.2–§4.4 for the full flow, sequence diagram, and attack mitigations. Summary of concrete parameters:

Parameter	Value
Password hashing	bcrypt, cost factor 12
Access token	JWT, HS256, 15 min expiry
Refresh token	Opaque random token, hashed at rest, 7 day expiry, rotated on use, httpOnly cookie
Token storage (client)	Access token in memory (React state/context) — never <code>localStorage/sessionStorage</code>

Parameter	Value
Logout	Deletes server-side refresh token row; client clears in-memory access token

20. Authorization (Consolidated)

See §4.1 for the full role/permission matrix and §13.1 for where `authenticate/authorize` sit in the middleware chain. Every route file explicitly declares its allowed roles at registration time, e.g.:

```
router.post('/work-orders', authenticate, authorize('ADMIN'), validate(createWorkOrderSchema), workOrdersController.create);
```

This makes the authorization requirement for every endpoint visible by reading the routes file alone — a reviewer (or the "explain your own application" live-verification step) can audit the entire permission surface in one file per module.

21. Transactions

Every operation below executes inside a single database transaction (Prisma `$transaction`), using `SERIALIZABLE`-safe patterns via explicit row locking (`SELECT ... FOR UPDATE`) rather than relying on Postgres's default `READ COMMITTED` isolation alone.

Operation	Reads	Locks	Writes	On Failure
Inventory adjustment	<code>inventory</code> row by id	<code>SELECT ... FOR UPDATE</code> on that row	update <code>physical_quantity</code> ; insert <code>inventory_transactions</code>	Whole transaction rolled back; no partial write. Idempotency key checked first (outside the row lock, via unique index) — a retry with the same key returns the cached result

Operation	Reads	Locks	Writes	On Failure
				instead of re-running.
Transfer — Dispatch	<code>internal_transfers</code> row (status), <code>inventory</code> row at source	<code>FOR UPDATE</code> on both	update <code>transfer.status</code> → <code>DISPATCHED</code> , set <code>dispatched_by/at</code> ; decrement source <code>physical_quantity</code> ; insert <code>inventory_transactions</code> (<code>TRANSFER_OUT</code>)	Rolled back entirely if source has insufficient available stock or transfer isn't <code>REQUESTED</code> .
Transfer — Receive	<code>internal_transfers</code> row, <code>inventory</code> row at destination (created if it doesn't yet exist for that item/location/batch)	<code>FOR UPDATE</code> on transfer row	conditional update <code>WHERE status='DISPATCHED'</code> → <code>RECEIVED</code> ; increment destination <code>physical_quantity</code> ; insert <code>inventory_transactions</code> (<code>TRANSFER_IN</code>)	If the conditional update affects 0 rows (already received / wrong status), the transaction is aborted and <code>409</code> returned — no inventory mutation ever happens on a failed receive attempt.
Reservation	<code>inventory</code> row for item/location/batch	<code>FOR UPDATE</code> on that row	increment <code>reserved_quantity</code> ; insert <code>reservations</code> row; insert <code>inventory_transactions</code> (<code>RESERVATION</code>)	Rolled back if <code>available_quantity</code> (re-read under the lock) is less than requested quantity → <code>409</code>

Operation	Reads	Locks	Writes	On Failure
				INSUFFICIENT_STOCK.
Customer order creation	—	—	insert customer_orders + customer_order_items	Simple insert transaction; does not reserve stock — reservation is a separate, explicit step per line (§17.5).
Work order status update	work_orders row	FOR UPDATE	conditional update based on current status + assignee	Rolled back on invalid transition or wrong assignee → 409/403.

Definitions, kept distinct as requested:

- **Database transaction** — an atomic set of reads/writes that either all commit or all roll back (the mechanisms in the table above).
- **Application-level validation** — shape/range checks on input before it ever touches the database (§18).
- **Authorization** — whether *this* user is allowed to attempt the operation at all (§20), checked before the transaction even opens.
- **Idempotency** — guaranteeing that *retrying the same logical request* (e.g. a client retry after a timeout) does not apply the effect twice, via a client-supplied idempotency key checked against a unique DB index (used for inventory adjustments; dispatch/receive get idempotency "for free" from their status-guarded conditional updates, since a retried dispatch/receive on an already-transitioned record simply returns the current state / a conflict rather than reapplying the mutation).

22. Concurrency

This is the section the case study weights most heavily in spirit ("Solve this correctly at backend/database level").

22.1 Chosen Strategy: Pessimistic Row Locking

For every operation that mutates an `inventory` row's `reserved_quantity` or `physical_quantity`, the service:

1. Opens a DB transaction.
2. Runs `SELECT ... FOR UPDATE` on the specific `inventory` row (by its primary key). Postgres blocks any other transaction trying to lock the *same row* until this one commits or rolls back.
3. Re-reads `available_quantity` from that locked row, not from any value fetched earlier in the request lifecycle.
4. Validates the requested quantity against that freshly-locked value.
5. Applies the update, inserts the audit row, commits (releasing the lock).

● **Why pessimistic over optimistic locking here:** optimistic locking (a `version` column checked on update, retried on conflict) works but pushes retry logic to the client/service caller and performs poorly under real contention (many retries when two users target the exact same hot inventory row, which is precisely the scenario the case study's Test 1 exercises). Pessimistic locking on a single, narrowly-scoped row is cheap here (transactions are short — a handful of statements — so lock hold time is small) and gives a simple, provably-correct guarantee: **the second transaction physically cannot read a stale "available" value**, because it cannot even acquire the row until the first transaction has committed its update. The `version` column is still included in the schema (§15.8) as a low-cost defense-in-depth / diagnostic aid, but the correctness guarantee does not depend on it.

22.2 Worked Example (Two Concurrent Reservations)

Available = 100

User A → Reserve 80 User B → Reserve 50 (arrive ~simultaneously)

T1 (User A's txn): BEGIN; SELECT inventory FOR UPDATE (row locked)

T2 (User B's txn): BEGIN; SELECT inventory FOR UPDATE → BLOCKS (waits for T1's lock)

T1: available (100) >= 80 → OK. reserved_quantity 0→80. INSERT reservation. COMMIT. (lock released)

T2: unblocks, now sees reserved_quantity = 80 → available = 20.

T2: available (20) >= 50 → FALSE → ROLLBACK, return 409 INSUFFICIENT_STOCK to User B.

Result: exactly one reservation succeeds; the other is correctly and safely rejected — never a partial or double reservation.

22.3 Other Concurrency Scenarios

Scenario	Handling
Two users dispatch the same transfer simultaneously	Both attempt <code>FOR UPDATE</code> on the transfer row; one wins, updates status to <code>DISPATCHED</code> ; the second, once unblocked, sees status is no longer <code>REQUESTED</code> and gets <code>409 INVALID_STATUS_TRANSITION</code> .
Two users receive the same transfer simultaneously	Same pattern: first to acquire the lock flips status to <code>RECEIVED</code> and increments destination inventory; the second's conditional update (<code>WHERE status='DISPATCHED'</code>) affects 0 rows → <code>409 TRANSFER_ALREADY_RECEIVED</code> , no double increment .
Transfer dispatch racing with an unrelated inventory adjustment at the same location	Both operations lock the same <code>inventory</code> row via <code>FOR UPDATE</code> ; Postgres serializes them — whichever acquires the lock first proceeds, the other sees the updated <code>physical_quantity</code> afterward. No special-casing needed beyond the row lock already in place.
General concurrent inventory writes	All inventory-mutating code paths (adjustment, dispatch, receive, reservation) go through the <i>same</i> lock-then-mutate pattern on the <code>inventory</code> table, so there is one, consistent concurrency primitive in the whole system rather than a different scheme per feature.

23. Business Rules / State Machines

23.1 Work Order

From	To	Trigger	Who
—	Assigned	Created	Admin
Assigned	In Progress	"Start" action	Operations User (must be the assignee)
In Progress	Completed	"Complete" action	Operations User (must be the assignee)
Assigned	Completed	✗ Invalid — must pass through In Progress	—
Completed	(any)	✗ Invalid — terminal state	—

stateDiagram-v2

[*] --> Assigned

Assigned --> InProgress: Start (assignee)

InProgress --> Completed: Complete (assignee)

Completed --> [*]

⚠ **AMBIGUITY:** the case study doesn't say whether *any* Operations User or *only the assigned* one may transition status. ● **RECOMMENDATION:** only the assigned Operations User (mirrors real shop-floor accountability — you shouldn't be able to mark someone else's work order complete). Easily relaxed to "any Operations User" by removing one condition in `WorkOrderService.updateStatus` if that's the intended reading.

23.2 Internal Transfer

From	To	Trigger	Who	Side Effect
—	Requested	Created	Operations User	none
Requested	Dispatched	"Dispatch"	Operations User	source physical_quantity -= quantity
Dispatched	Received	"Receive"	Operations User	destination physical_quantity += quantity
Requested	Received	✗ Invalid — must dispatch first	—	—
Dispatched	Dispatched (re-dispatch)	✗ Invalid	—	—
Received	(any)	✗ Invalid — terminal state	—	—

stateDiagram-v2

[*] --> Requested

Requested --> Dispatched: Dispatch (Operations)

Dispatched --> Received: Receive (Operations)

Received --> [*]

Explicit failure-mode answers (as required by the case study):

- **Transfer quantity exceeds source available quantity:** rejected at creation time (400 INSUFFICIENT_STOCK) and re-validated at Dispatch time in case stock changed in between (400 INSUFFICIENT_STOCK again, transfer stays Requested).
- **Destination receives twice:** structurally prevented — see §21/§22.3 (conditional update + status check); second attempt gets 409 TRANSFER_ALREADY_RECEIVED.

- **Invalid status transition (e.g. Receive before Dispatch): 409**
`INVALID_STATUS_TRANSITION`, no inventory mutation occurs.
- **Transaction fails mid-way (e.g. DB connection drop after decrementing source but before the transaction commits):** the whole DB transaction rolls back atomically — Postgres guarantees the decrement is undone too; there is no intermediate persisted state where source is decremented but the transfer still shows `Requested`, because both writes happen in one transaction.

23.3 Customer Order Item

From	To	Trigger	Who
—	Pending	Order created	Sales User
Pending	Reserved	"Reserve"	Sales User
Reserved	(any)	✗ Invalid today — cancellation/release is a \$35 future extension	—

24. Error Handling

24.1 Standard Error Response

```
{
  "success": false,
  "error": {
    "code": "INSUFFICIENT_STOCK",
    "message": "Insufficient available inventory",
    "details": {}
  }
}
```

24.2 Error Code → HTTP Status Mapping

Code	HTTP Status	Meaning
VALIDATION_ERROR	400	Request body/params fail schema validation.
INVALID_CREDENTIALS	401	Login failed.
UNAUTHENTICATED	401	Missing/invalid/expired access token.
FORBIDDEN	403	Authenticated but role/ownership doesn't permit this action.
NOT_ASSIGNED_TO_WORK_ORDER	403	Operations User is not the WO's assignee.
NOT_FOUND	404	Referenced entity does not exist.
SAME_SOURCE_AND_DESTINATION	400	Transfer source = destination.
INSUFFICIENT_STOCK	409	Requested quantity exceeds available (reservation, transfer create/dispatch).
INVALID_STATUS_TRANSITION	409	State machine violation.
TRANSFER_ALREADY_RECEIVED	409	Duplicate receive attempt.
DUPLICATE_TRANSACTION	409	Idempotency key reuse.
DUPLICATE_INVENTORY_RECORD	409	Unique constraint on item/location/batch violated.
RATE_LIMITED	429	Login brute-force guard tripped.

Code	HTTP Status	Meaning
INTERNAL_ERROR	500	Unexpected server/database failure (never leaks internals to the client).

A single Express `errorHandler` middleware maps a small set of typed application error classes (`ValidationError`, `NotFoundError`, `ConflictError`, `ForbiddenError`, etc., thrown from services) to this format, so controllers never manually build error JSON.

25. Security

Area	Measure
Authentication	JWT + rotated refresh tokens, bcrypt password hashing (§19).
Authorization	Server-enforced role middleware on every mutating route, no frontend-only gating (§20).
Password security	bcrypt cost 12; never logged, never returned in any API response.
Token security	Access token in memory only; refresh token <code>httpOnly/Secure/SameSite=Strict</code> cookie; rotation + family revocation on reuse detection.
Rate limiting	<code>express-rate-limit</code> on <code>/auth/login</code> and <code>/auth/refresh</code> .
CORS	Explicit allow-list via <code>CORS_ORIGIN</code> env var; credentials enabled only for the known frontend origin.
CSRF	Mitigated by <code>SameSite=Strict</code> cookie + requiring the <code>Authorization</code> header (not derivable cross-site) for all state-changing requests.

Area	Measure
SQL injection	Fully parameterized via Prisma; no raw string-concatenated SQL anywhere in the codebase.
XSS	React escapes output by default; no <code>dangerouslySetInnerHTML</code> usage; CSP header set.
Input validation	Zod schemas on every request body/query/param, both client and server (§18).
Secure headers	<code>helmet</code> middleware (HSTS, X-Content-Type-Options, X-Frame-Options, CSP).
Secrets	All secrets via environment variables, never committed; <code>.env.example</code> documents required vars without real values (§29).
Logging	No sensitive data (passwords, tokens) in logs; request IDs for traceability.
Audit trail	<code>inventory_transactions</code> records who/what/when for every inventory change (§26).
Sensitive info exposure	Generic error messages to clients for 5xx errors; stack traces never sent to the client, only to server logs.

26. Audit & Inventory Transaction History

Every inventory-affecting operation writes one row to `inventory_transactions` (schema in §15.9) in the *same* database transaction as the mutation itself — never as a fire-and-forget side effect — so the audit trail can never drift out of sync with actual stock levels (if the audit insert fails, the whole operation rolls back).

This is useful because:

- **Debugging:** if `available_quantity` ever looks wrong, the full before/after history for that inventory row answers "what changed it and when" without guessing.
- **Operational traceability:** every transfer dispatch/receive and every reservation is individually attributable to a user and a reference entity (`reference_type/reference_id`), satisfying real-world audit needs (who moved this stock, and why) without building a separate reporting module.
- **Live-verification readiness:** if "Damaged stock" (§35, Change 1) is added later, it's simply a new `transaction_type` value plus one new endpoint — the audit model doesn't need to change.

27. Testing Strategy

27.1 Layers

Layer	Tool	Scope
Unit	Vitest	Pure service logic (shortage calculation, state-machine validity checks) with a mocked repository.
Service/DB integration	Vitest + a real test Postgres (via Docker/Testcontainers)	Services against a real database, including transaction/locking behavior — required for the concurrency tests, which cannot be meaningfully mocked.
API	Vitest + Supertest	Full HTTP request/response cycle including auth middleware, per endpoint.
Authorization	Supertest	Every protected endpoint hit with wrong/no role → expect 401/403.
Frontend component	Vitest + React Testing Library	Individual components (forms, status badges, tables) in isolation.

Layer	Tool	Scope
Frontend integration	React Testing Library + MSW (mocked API)	Page-level flows (e.g. filling the Create Transfer form and submitting).
End-to-end	Playwright	The 5-screen happy path end-to-end against a running backend + test DB.

27.2 Test Case Table (includes the 5 mandatory cases)

Test ID	Scenario	Preconditions	Action	Expected Result	Layer
T-01	Cannot reserve more than available inventory	Inventory available = 50	Sales User reserves 60	409 INSUFFICIENT_STOCK, reserved_quantity unchanged	Service/DB integration
T-02	Cannot transfer more than available inventory	Source available = 20	Operations User creates transfer for 30	400 INSUFFICIENT_STOCK, no transfer row committed (or committed but blocked at dispatch — see note)	API
T-03	Destination stock increases only after receipt	Transfer created + dispatched	Check destination inventory before "receive" is called	Destination physical_quantity unchanged until receive; increases exactly by quantity after receive	Service/DB integration

Test ID	Scenario	Preconditions	Action	Expected Result	Layer
T-04	Same transfer cannot be received twice	Transfer status = RECEIVED	Call PATCH /transfers/:id/receive again	409 TRANSFER_ALREADY_RECEIVED, destination inventory unchanged on the 2nd call	API
T-05	Unauthorized user cannot perform restricted operation	Logged in as Sales User	POST /work-orders	403 FORBIDDEN	API / Authorization
T-06	Concurrent reservations cannot jointly oversell	Available = 100	Fire two simultaneous POST /orders/:id/reserve for 80 and 50	Exactly one succeeds (200), the other fails (409 INSUFFICIENT_STOCK); final reserved_quantity reflects only the successful one	Service/DB integration (concurrency)
T-07	Available quantity always equals physical – reserved	Any sequence of adjustments/reservations	Run a randomized sequence of operations	available_quantity (DB-generated) matches physical – reserved after every step	Unit + DB

Test ID	Scenario	Preconditions	Action	Expected Result	Layer
T-08	Negative inventory is rejected	physical_quantity = 10	Adjustment of -15	400 VALIDATION_ERROR (or DB CHECK violation surfaced as 400), physical_quantity unchanged	API
T-09	Duplicate inventory transaction is rejected	Adjustment submitted with idempotency key X	Same request replayed with key X	Second call returns the original result, no second inventory_transactions row inserted	Service/DB integration
T-10	Work order shortage calculated correctly	Required = 100, Available at location = 60	GET /work-orders/:id/stock-check	{ shortage: 40 }	Unit
T-11	Invalid work order status transition rejected	Status = Completed	PATCH .../status { status: 'IN_PROGRESS' }	409 INVALID_STATUS_TRANSITION	API
T-12	Non-assigned cannot advance work order	WO assigned to User B	User A (Operations) attempts to advance status	403 NOT_ASSIGNED_TO_WORK_ORDER	API
T-13	Transfer source ≠	—	Create transfer with	400 SAME_SOURCE	API

Test ID	Scenario	Preconditions	Action	Expected Result	Layer
	destination enforced		same source/destination	E_AND_DESTINATION	
T-14	Frontend: Reserve button disabled after reservation	Order line already reserved	Render OrderDetail	"Reserve" control not shown/disabled for that line	Frontend component
T-15	Frontend: reservation-conflict error surfaces inline	MSW mocks 409 INSUFFICIENT_STOCK	Submit Reserve	Inline error shown with server message, form not cleared	Frontend integration
T-16	E2E happy path	Seeded data	Login → Inventory → create WO → stock check → create+dispatch+receive transfer → create order → reserve	All steps succeed, final inventory numbers are correct	E2E

Concurrency tests (T-06) are run against a **real** database with actual parallel connections (not mocked), since the correctness claim in §22 depends on genuine Postgres row-locking behavior, which cannot be faithfully simulated with a mocked repository.

28. Performance

- **Indexing:** every FK column used in a **WHERE/JOIN** is indexed (see each table definition in §15); **inventory(item_id, location_id)** composite index supports the most common lookup (stock check, reservation).

- **Pagination:** all list endpoints are server-paginated (default 20, max 100) — no unbounded `SELECT *`.
- **N+1 avoidance:** Prisma `include/select` used to fetch related item/location/batch names in the same query as the inventory list, not per-row follow-up queries.
- **Response size:** list endpoints return only the fields the relevant page needs (DTOs, not raw Prisma models).
- **Connection pooling:** Prisma's built-in connection pool, sized via `DATABASE_URL` connection-limit parameter appropriate to the deployment target.
- **Transaction duration:** kept intentionally short (a handful of statements, no external I/O or network calls inside a locked transaction) to minimize lock contention under load.
- **Caching:** client-side only (TanStack Query); no server-side cache for mutable data, since staleness here directly risks incorrect stock decisions —  correctness prioritized over the marginal performance gain at this scale.
- **Logging overhead:** structured, asynchronous logging (pino); no synchronous console logging in hot paths.

29. Configuration & Environment

Variable	Used By	Example	Notes
<code>NODE_ENV</code>	Backend	<code>development / test / production</code>	Controls logging verbosity, error detail exposure, Swagger UI availability.
<code>PORT</code>	Backend	<code>4000</code>	
<code>DATABASE_URL</code>	Backend	<code>postgresql://user:pass@localhost:5432/erp</code>	Prisma connection string; different value per environment.
<code>JWT_SECRET</code>	Backend	(random 256-bit string)	Access-token signing key.
<code>JWT_EXPIRES_IN</code>	Backend	<code>15m</code>	
<code>REFRESH_TOKEN_SECRET</code>	Backend	(random 256-bit string)	Used if refresh tokens are also signed rather than pure-random-plus-hash ( either

Variable	Used By	Example	Notes
			approach is acceptable; this spec uses random+hash, so this var is optional depending on chosen implementation).
REFRESH_TOKEN_EXPIRES_IN	Backend	7d	
CORS_ORIGIN	Backend	http://localhost:5173	Exact allow-listed frontend origin(s).
LOG_LEVEL	Backend	info	pino level.
VITE_API_BASE_URL	Frontend	http://localhost:4000/api/v1	

All variables are documented in `.env.example` with placeholder values (never real secrets committed). §13.3 requires the backend to validate this set at boot via Zod and fail fast if anything required is missing — this is what makes the system "portable" per the case study's explicit requirement: nothing is hardcoded to a specific host/provider.

30. Tech Stack

Layer	Technology	Purpose	Reason for Selection
Frontend framework	React 18 + TypeScript	UI	Case-study-aligned; strong ecosystem; type safety end-to-end with the backend.
Build tool	Vite	Dev server/bundler	Fast HMR, minimal config.
Routing	React Router v6	Client-side routing	De facto standard for React SPAs.

Layer	Technology	Purpose	Reason for Selection
Server state	TanStack Query	Data fetching/caching/invalidation	Purpose-built for exactly the fetch/cache/invalidate pattern this app needs; avoids hand-rolled loading/error state.
Forms	React Hook Form + Zod	Form state + validation	Minimal re-renders, schema reusable to mirror backend validation.
Styling	Tailwind CSS	Utility-first styling	Fast to build a consistent, dense ERP UI without a heavy component library.
HTTP client	Axios	API calls	Interceptors for auth header injection and 401→refresh handling.
Backend runtime	Node.js + TypeScript	API server	Case-study-aligned; one language across the stack.
Backend framework	Express	REST routing/middleware	Lightweight, explicit, easy to reason about the middleware chain (§13.1).
Database	PostgreSQL	Relational data store	 required to be relational; Postgres specifically chosen for row-level locking (<code>SELECT FOR UPDATE</code>), generated columns, and check constraints — all

Layer	Technology	Purpose	Reason for Selection
			used directly in this design (§14, §21, §22).
ORM	Prisma	Data access + migrations	Type-safe queries matching the TypeScript stack, first-class migration tooling, supports raw <code>FOR UPDATE</code> queries where needed via <code>\$queryRaw</code> /interactive transactions.
Auth	jsonwebtoken + bcrypt	Token signing, password hashing	Industry-standard, minimal dependencies.
Testing (backend)	Vitest, Supertest	Unit/integration/API tests	Fast, TypeScript-native.
Testing (frontend)	Vitest, React Testing Library, MSW	Component/integration tests	Standard React testing stack.
Testing (E2E)	Playwright	Full browser flow tests	Reliable cross-browser automation.
API docs	OpenAPI 3.0 + Swagger UI	API documentation	Explicit contract, satisfies submission requirement.
Logging	pino	Structured logs	Low overhead, JSON output suited to any log aggregator (portability).
Containerization	Docker + Docker Compose	Local/dev parity, portability	Runs identically anywhere Docker runs — directly supports the "not tied

Layer	Technology	Purpose	Reason for Selection
			to one hosting provider" requirement.

31. Project Folder Structure (Top Level)

mini-ops-erp/

```

├── frontend/      # see §8.3
├── backend/       # see §13.2
├── docs/
|   ├── er-diagram.png
|   └── demo-video-link.md
├── docker-compose.yml # postgres + backend + frontend for local dev
├── .github/workflows/ # CI (lint, typecheck, test) — see §36
└── README.md

```

32. Development Workflow / Implementation Order

1. **Project setup** — monorepo scaffolding, Docker Compose (Postgres), linting/formatting config.
2. **Database setup** — Prisma schema (§15), initial migration, seed script.
3. **Authentication** — login, JWT issuance, refresh rotation, logout (§19). *Depends on: users/roles tables.*
4. **Authorization middleware** — `authenticate/authorize` (§20). *Depends on: 3.*
5. **Inventory module** — items/categories/locations/inventory CRUD + adjustment endpoint (§15.6–15.9, §17.2). *Depends on: 4.*

6. **Work orders** — creation + stock-check + status transitions (§17.3, §22.1, §23.1).
Depends on: 5 (reads inventory for stock check).
 7. **Stock checking logic** — shared shortage-calculation utility used by Work Orders and (optionally) the Transfer-creation shortcut. *Depends on: 5.*
 8. **Internal transfers** — create/dispatch/receive with locking (§17.4, §21, §22.3, §23.2).
Depends on: 5.
 9. **Customer orders & reservations** — order creation + atomic reservation (§17.5, §22.2, §23.3). *Depends on: 5.*
 10. **Transaction & concurrency hardening pass** — add/verify row-locking on every mutation path once all modules exist; write the concurrency test suite (T-06) against the real flows. *Depends on: 6, 8, 9.*
 11. **Frontend integration** — build the 5 pages against the now-stable API (§8–§9). *Can start in parallel from step 5 onward using a mocked API (MSW), then swap to the real backend incrementally.*
 12. **Testing** — fill out the full test matrix (§27), especially the 5 mandatory scenarios.
Ongoing from step 3, finalized at the end.
 13. **Documentation** — README, OpenAPI spec finalization, ER diagram export (§36).
 14. **Deployment / packaging** — Docker Compose validated end-to-end; environment variable documentation finalized.
-

33. End-to-End User Flows

Flow 1 — Login → Inventory

Actor: any user. Frontend: submit login form. API: `POST /auth/login`. Backend: `AuthenticationService` verifies credentials, issues tokens. DB: reads `users`, inserts `refresh_tokens`. Response: tokens + user. UI: redirect to `/inventory`, `GET /inventory` fires, table renders. Errors: invalid credentials → inline banner; rate-limited → banner with retry-after messaging.

Flow 2 — Admin creates Work Order → Stock Check → Shortage

Actor: Admin. Frontend: "Create Work Order" modal submit. API: `POST /work-orders`. Backend: `WorkOrderService` validates assignee role, creates row. DB: insert into `work_orders`. Response: 201 + WO. UI: row appears, status badge "Assigned." Actor then opens Stock Check drawer → `GET /work-orders/:id/stock-check` → `StockCheckService` reads `inventory` for that location/item, computes shortage → UI renders Required/Available/Shortage. Errors: assignee not an Operations User → 400; location/item not found → 404.

Flow 3 — Operations creates Transfer → Dispatch → Inventory decrease → Receive → Destination increase

Actor: Operations User. Frontend: Create Transfer modal → `POST /transfers` (status `Requested`). Then "Dispatch" → `PATCH /transfers/:id/dispatch: TransferService` locks transfer+source inventory rows, validates `available ≥ quantity`, decrements source, logs `TRANSFER_OUT`, commits. UI: badge → "Dispatched," inventory list (if open) refetches and shows reduced source quantity. Then "Receive" → `PATCH /transfers/:id/receive: TransferService` locks transfer row, conditional update to `Received`, increments destination inventory, logs `TRANSFER_IN`, commits. UI: badge → "Received," destination inventory refetches and shows increased quantity. Errors at each step per §23.2's explicit failure-mode table.

Flow 4 — Sales creates Customer Order → Reserve → Reserved↑ / Available↓

Actor: Sales User. Frontend: Create Order modal → `POST /orders` (line items `Pending`). Then "Reserve" per line → `POST /orders/:id/reserve: ReservationService` locks the target `inventory` row, re-checks `available` under the lock, increments `reserved_quantity`, inserts `reservations` row, logs `RESERVATION`, commits. UI: order line badge → "Reserved," inventory available quantity refetches and drops accordingly. Errors: insufficient stock → `409`, shown inline on that line.

Flow 5 — Concurrent Reservation Conflict

Actors: Sales User A and Sales User B, same inventory row, nearly simultaneous "Reserve" clicks. Frontend: both send `POST /orders/:id/reserve` (different orders, same inventory). Backend: both requests hit `ReservationService`; both attempt `SELECT ... FOR UPDATE` on the same `inventory` row; Postgres serializes them (§22.2). The first to acquire the lock succeeds and commits; the second, once unblocked, re-reads the now-updated `reserved_quantity`, finds `available` insufficient, and rolls back, returning `409 INSUFFICIENT_STOCK`. UI (loser): inline error "Only N units are now available," quantity field highlighted, available-quantity hint refreshed from the server. UI (winner): reservation confirmed normally.

flowchart LR

A[Frontend Action] --> B[API Endpoint]

B --> C{authenticate}

C -->|401| Z1[Reject]

C --> D{authorize role}

D -->|403| Z2[Reject]

D --> E[validate schema]

E -->|400| Z3[Reject]

E --> F[Service: business rules]

F --> G[DB Transaction: lock -> validate -> mutate -> log]

G -->|conflict| Z4[409 rollback]

G -->|ok| H[Commit]

H --> I[Response]

I --> J[Frontend: update cache, toast, re-render]

34. Edge Cases

Edge Case	Handling
Quantity = 0	Rejected — all quantity fields require <code>> 0</code> (<code>CHECK</code> constraint + Zod <code>.positive()</code>).
Negative quantity	Rejected at validation layer and by <code>CHECK</code> (<code>>= 0 / > 0</code>) constraints as a backstop.
Decimal quantity where unsupported	<code>items.allow_fractional = false</code> (default) forces integer validation in Zod (<code>Number.isInteger</code>); items explicitly flagged fractional (e.g. liquids by volume) allow up to 3 decimal places per the <code>numeric(14,3)</code> column type.
Item does not exist	<code>404 NOT_FOUND</code> from FK-existence check before any write.
Location does not exist	<code>404 NOT_FOUND</code> , same pattern.

Edge Case	Handling
Batch does not exist	404 NOT_FOUND ; if omitted, defaults to the item's DEFAULT batch (§15.7).
Inventory record does not exist for a requested item/location/batch	404 NOT_FOUND on read; on write (adjustment), the record must be created first via POST /inventory (explicit, not implicitly auto-vivified, to avoid silently creating phantom stock rows).
Insufficient stock	409 INSUFFICIENT_STOCK (reservation/dispatch) or 400 INSUFFICIENT_STOCK (creation-time pre-check) — see §17 for which code path uses which status, both carry the same error code.
Duplicate transfer (double-submit of "Create Transfer")	Not idempotency-keyed by default (creating two legitimate transfers with identical fields is a valid business scenario), but the frontend disables the submit button after first click to avoid accidental double network calls; if truly needed, an idempotency key can be added the same way as inventory adjustments.
Duplicate receive	Structurally prevented (§21/§22.3) — 409 TRANSFER_ALREADY_RECEIVED .
Already-completed work order receives another status change	409 INVALID_STATUS_TRANSITION — Completed is terminal.
Invalid work order status change (e.g. skip to Completed)	409 INVALID_STATUS_TRANSITION .
Invalid transfer status change	409 INVALID_STATUS_TRANSITION .
Reservation race condition	See §22.2 — resolved via row locking, loser gets 409 .
Network request retry (client re-sends after timeout, server actually processed it)	Reads are safely idempotent by nature; the one write endpoint with a genuine "did this already happen?" risk (inventory adjustment) uses an idempotency key (§21);

Edge Case	Handling
	dispatch/receive are naturally idempotent-safe because a retry lands on an already-transitioned status and gets a 409 rather than reapplying.
Database transaction failure mid-operation	Entire transaction rolls back atomically (Postgres guarantee) — no partial state is ever persisted.
Partial request failure (e.g. multi-line order reservation, line 2 of 3 fails)	Each line's reservation is its own independent API call/transaction (§17.5) — lines 1 and 3 remain reserved if they succeeded; line 2 shows its specific error. The order header status reflects a mixed state (PARTIALLY_RESERVED) rather than silently rolling back unrelated successful lines.
Unauthorized access	401/403 per §20, enforced server-side regardless of what the frontend shows.
User attempts to access another location's data	Not restricted today (⚠ no location-scoping requirement in the base case study) — this is precisely anticipated future Change 4 (§35); the users.location_id column and a single new authorization check are the intended extension point.
Deleted/inactive records	Soft-deactivation only (is_active = false on items/locations/users) — nothing is hard-deleted, since FKs (ON DELETE RESTRICT) protect referential integrity and history (work orders, transfers, transactions) must remain queryable forever. Inactive records are excluded from selection dropdowns but remain visible in historical records/reports.
Duplicate inventory transaction	Prevented via idempotency key uniqueness (§21) for adjustments; structurally impossible for dispatch/receive (status-guarded).

35. Extensibility / Live-Verification Readiness

The case study warns that shortlisted candidates receive **one small unannounced change**. The architecture is deliberately shaped so each of the four listed examples is a localized change, not a rewrite:

Anticipated Change	What Already Supports It	What Would Actually Change
1. "Damaged" status reduces available stock automatically	<code>inventory_transactions.transaction_type</code> is an open string, not a closed enum baked into multiple places; <code>available_quantity</code> is a single generated formula.	Add DAMAGED as a valid <code>transaction_type</code> ; add a <code>damaged_quantity</code> column (or reuse a negative ADJUSTMENT) and, if "available" should subtract damaged stock separately from reservations, extend the generated-column formula to <code>physical - reserved - damaged</code> . One migration, one new service method, no changes to Transfer/Reservation logic.
2. Partial transfer receipt	<code>internal_transfers.quantity</code> is already separate from what's been received so far conceptually; the receive handler is already an isolated, lockable operation.	Add a <code>received_quantity</code> column (replacing the implicit "all or nothing" assumption), allow PATCH <code>/transfers/:id/receive</code> to accept a partial <code>quantity</code> body, keep status DISPATCHED until <code>received_quantity == quantity</code> , then flip to RECEIVED . The locking pattern (§22.1) is unchanged.
3. Cancel an order and release reserved inventory	<code>reservations.status</code> already includes RELEASED as a defined enum value (§15.14) specifically for this reason;	Add POST <code>/orders/:id/cancel</code> (or per-line <code>.../items/:id/cancel</code>): inside a locked transaction,

Anticipated Change	What Already Supports It	What Would Actually Change
	<code>customer_order_items.status</code> already includes <code>CANCELLED</code> .	decrement the matching inventory's <code>reserved_quantity</code> , set reservation status to <code>RELEASED</code> , log an <code>RESERVATION_RELEASE</code> transaction. No schema change needed — the enum values were already reserved for this.
4. Restrict users to their assigned location	<code>users.location_id</code> already exists (nullable) for exactly this purpose (§15.2).	Add a check in the relevant authorization layer/service: if <code>user.location_id</code> is set, filter/reject operations on <code>inventory/work_orders/transfers</code> where the location doesn't match. A single new middleware or service-level guard, applied per-module; no schema migration.

General extensibility principles applied throughout: status fields are open strings validated by `CHECK` constraints (easy to extend the allowed set via a migration) rather than native Postgres `ENUM` types (which are more painful to alter); all business rules live in the Service layer behind a stable Controller/Route interface, so a rule change never requires touching routing, auth, or the frontend's data-fetching plumbing; the audit table (`inventory_transactions`) already has an open `transaction_type` and `reference_type` design that new features plug into rather than needing their own parallel audit mechanism.

36. Documentation & Submission

36.1 Documentation Checklist

- ☐ Git repository with **incremental commit history** (not one final commit) — commits roughly following the implementation order in §32.

- ☐ README containing: tech stack, project setup, database setup (migration + seed commands), environment variables (table from §29), how to run (dev + Docker), how to run tests.
- ☐ Database schema / ER diagram (exported from the Mermaid diagram in §15.15, or generated via [prisma-erd-generator](#)).
- ☐ API documentation — OpenAPI/Swagger served at [/docs](#), or an exported Postman collection.
- ☐ Short demo video (5–7 minutes max).

36.2 Demo Video Flow

1. **Login** — show logging in as each of the three roles briefly, highlighting the role-based nav differences.
2. **Inventory** — show the table, an Operations User adjusting stock, the Physical/Reserved/Available numbers updating.
3. **Work Order** — Admin creates a Work Order; show the Stock Check drawer and the calculated shortage.
4. **Transfer** — Operations User creates a transfer covering the shortage, dispatches it (show source inventory drop), receives it (show destination inventory rise).
5. **Order Reservation** — Sales User creates a Customer Order and reserves stock; show Available dropping accordingly. Optionally show the concurrency conflict by triggering two reservations quickly to demonstrate the [409 INSUFFICIENT_STOCK](#) guard.

37. Observability & Maintainability

Practice	Detail
Structured logging	pino, JSON lines, one entry per request.
Request/correlation IDs	Middleware assigns a UUID per request, included in every log line and echoed back as an X-Request-Id response header — makes it possible to trace a single user action across log lines.
Audit logs	inventory_transactions (business audit) + application logs (technical audit) are kept separate and serve different purposes.
Monitoring	Out of scope to stand up a full stack for this case study; the health-check endpoint below

Practice	Detail
	is the minimum hook a real monitor would poll.
Health check	<code>GET /health</code> — no auth required, returns <code>{ status: 'ok', db: 'ok' }</code> after a trivial DB ping; used for container orchestration liveness checks.
Graceful shutdown	<code>server.close()</code> on <code>SIGTERM/SIGINT</code> , draining in-flight requests and closing the Prisma connection pool before exit.
Migrations	Prisma Migrate, versioned files committed to git; <code>prisma migrate deploy</code> in production, <code>prisma migrate dev</code> locally.
Seed data	<code>prisma/seed.ts</code> , idempotent (safe to re-run), run via <code>npm run seed</code> .
API versioning	<code>/api/v1</code> prefix from day one, so a future breaking change can ship as <code>/api/v2</code> alongside it.
Linting/formatting	ESLint + Prettier, shared config across frontend/backend.
Type safety	Strict TypeScript (<code>strict: true</code>) on both frontend and backend; Prisma-generated types shared conceptually via a mirrored <code>types/</code> folder or a shared package if the monorepo tooling supports it.
CI/CD	GitHub Actions: on every PR — install, lint, typecheck, run unit+integration tests against a throwaway Postgres service container.
Git conventions	Conventional commits (<code>feat:</code> , <code>fix:</code> , <code>test:</code> , <code>chore:</code>), one logical change per commit, satisfying the "reasonable development history" submission requirement.

Practice	Detail
PR checks	CI must pass before merge (even in a solo project, this documents intent and gives a clean CI log for review).
Dependency management	Lockfiles committed (<code>package-lock.json</code>), <code>npm audit</code> run in CI (informational, non-blocking at this scale).

38. Requirements Traceability Matrix

Req ID	Requirement	Frontend	Backend	Database	API	Test	Status
REQ-01	Auth with 3 roles	Login page (§9.1)	AuthenticationService (§16)	users, roles (§15.1–15.2)	<code>/auth/*</code> (§17.1)	—	Planned
REQ-02	Backend authorization mandatory	Role-filtered nav, RoleGuard (§8.4)	authenticate/authenticate middleware (§20)	roles	every mutating route	T-05	Planned
REQ-03	Inventory mgmt (item/category/location/batch/qty)	Inventory page (§9.2)	Inventory Service (§16)	items, categories, locations, batches, inventory (§15.4–15.8)	<code>/inventory/*</code>	T-07	Planned
REQ-04	Available = Physical – Reserved	StockAvailabilityIndicator	Inventory Service, ReservationService	<code>available_quantity</code> generated column (§14.1)	<code>/inventory</code>	T-07	Planned

Req ID	Requirement	Frontend	Backend	Database	API	Test	Status
	, always correct						
REQ-05	Prevent negative inventory	Client-side qty validation	Inventory Service validation	CHECK constraints (§15.8)	/inventory/:id PATCH	T-08	Planned
REQ-06	Prevent invalid quantity	Quantity input component	Zod schemas (§18)	CHECK (> 0)	all write endpoints	T-08	Planned
REQ-07	Prevent duplicate inventory transaction	idempotency key generated per submit	Inventory Service	unique idempotency_key index (§15.9)	/inventory/:id PATCH	T-09	Planned
REQ-08	Prevent reservation beyond available	live available hint	ReservationService row-locking (§22.2)	CHECK (available_quantity >= 0)	/orders/:id/reserve	T-01, T-06	Planned
REQ-09	Admin creates Work Order w/ required fields	Create WO modal (§9.3)	WorkOrderService	work_orders (§15.10)	POST /work-orders	—	Planned
REQ-10	Work Order status lifecycle	WorkOrderStatus Component	state-machine check (§23.1)	CHECK (status IN ...)	PATCH /work-orders/:id/status	T-11, T-12	Planned
REQ-11	Automatic material	Stock Check drawer	StockCheckService (§16)	reads inventory	GET /work-orders/:	T-10	Planned

Req ID	Requirement	Frontend	Backend	Database	API	Test	Status
	shortage calculation				id/stock-check		
REQ-12	Internal Transfer creation & fields	Create Transfer modal (§9.4)	Transfer Service	internal_transfers (§15.11)	POST /transfers	T-13	Planned
REQ-13	Transfer status lifecycle	Transfer StatusComponent	state-machine check (§23.2)	CHECK (status IN ...)	PATCH /transfers/:id/dispatch,/receive	—	Planned
REQ-14	Dispatch decreases source	Dispatch confirmation dialog	Transfer Service.dispatch (§21)	inventory.physical_quantity	PATCH .../dispatch	T-03	Planned
REQ-15	Destination unaffected before receipt	—	Transfer Service.dispatch does not touch destination	—	—	T-03	Planned
REQ-16	Receipt increases destination	Receive confirmation dialog	Transfer Service.receive (§21)	inventory.physical_quantity	PATCH .../receive	T-03	Planned
REQ-17	Prevent duplicate receive	Receive button hidden post-receipt	conditional update + status guard (§21, §22.3)	CHECK (status IN ...)	PATCH .../receive	T-04	Planned

Req ID	Requirement	Frontend	Backend	Database	API	Test	Status
REQ-18	Sales creates Customer Order	Create Order modal (§9.5)	CustomerOrderService	customer_orders, customer_order_items (§15.12–15.13)	POST /orders	—	Planned
REQ-19	Concurrency-safe reservation	Inline conflict handling (§9.5)	ReservationService row-locking (§22.2)	reservations, FOR UPDATE	POST /orders/:id/reserve	T-01, T-06	Planned
REQ-20	Frontend connected to backend APIs	API client (§8.3, §12)	REST controllers	—	all endpoints	T-16	Planned
REQ-21	Relational database	—	Prisma	PostgreSQL schema (§15)	—	—	Planned
REQ-22	Proper data relationships	—	—	FKs across all tables (§14.2)	—	—	Planned
REQ-23	Backend validation	Mirrored client Zod schemas	Zod schemas server-side (§18)	CHECK/NOT NULL/UNIQUE	all write endpoints	—	Planned
REQ-24	Error handling	Error banners/toasts (§8.7)	errorHandler middleware (§24)	—	standard error envelope	—	Planned

Req ID	Requirement	Frontend	Backend	Database	API	Test	Status
REQ-25	Transactions where required	—	Prisma \$transaction + FOR UPDATE (§21)	—	—	T-01–T-06	Planned
REQ-26	Environment-based config / portability	VITE_API_BASE_URL	Zod-validated env config (§29)	DATABASE_URL	—	—	Planned
REQ-27	Screen: Login	§9.1	—	—	/auth/login	—	Planned
REQ-28	Screen: Inventory	§9.2	—	—	/inventory	—	Planned
REQ-29	Screen: Work Orders	§9.3	—	—	/work-orders	—	Planned
REQ-30	Screen: Internal Transfers	§9.4	—	—	/transfers	—	Planned
REQ-31	Screen: Customer Orders	§9.5	—	—	/orders	—	Planned
REQ-32	Test: reservation limit	—	—	—	—	T-01	Planned
REQ-33	Test: transfer limit	—	—	—	—	T-02	Planned
REQ-34	Test: destination	—	—	—	—	T-03	Planned

Req ID	Requirement	Frontend	Backend	Database	API	Test	Status
	increases only after receipt						
REQ-35	Test: no double receive	—	—	—	—	T-04	Planned
REQ-36	Test: unauthorized blocked	—	—	—	—	T-05	Planned
REQ-37	Git history, README, ER diagram, API docs, demo video	—	—	—	—	—	Planned (§36)
REQ-38	Extensibility for live-verification change	—	Open transaction_type, service-layer isolation (§35)	reserved enum values (RELEASED, PARTIALLY_RESERVED), nullable location_id	—	—	Planned (§35)

(Status column is "Planned" for every row because this document is the pre-implementation specification; update to "Implemented"/"Tested" as work lands — this table is meant to be kept alive throughout development, not archived after being written once.)

39. Final Architecture Summary

A. High-Level Architecture

flowchart TB

subgraph Client

FE[React SPA]

end

subgraph Server

API[Express REST API]

MW[Auth + Authz Middleware]

SVC[Service Layer]

REPO[Prisma Repository Layer]

end

DB[(PostgreSQL)]

FE -- HTTPS / JSON --> API

API --> MW --> SVC --> REPO --> DB

B. Frontend Architecture

React 18 + TypeScript SPA, feature-folder structure, TanStack Query for all server state, React Hook Form + Zod for forms, role-aware routing via [ProtectedRoute/RoleGuard](#) (§8–§9).

C. Backend Architecture

Layered Express app: Route → authenticate → authorize → validate → Controller → Service → Repository (Prisma) → Postgres. Business rules live exclusively in the Service layer (§13, §16).

D. Database Architecture

PostgreSQL, normalized relational schema, `available_quantity` as a database-generated column, `CHECK` constraints as a structural backstop against invalid states, full audit trail via `inventory_transactions` (§14–§15, §26).

E. Authentication Architecture

JWT access tokens (15 min, in-memory) + rotated opaque refresh tokens (7 day, httpOnly cookie), bcrypt password hashing (§19).

F. Authorization Architecture

Server-enforced RBAC via `authorize(...roles)` middleware declared per route; no business-critical check ever lives only in the frontend (§20).

G. Transaction Architecture

Every multi-step mutation (adjust, dispatch, receive, reserve) runs inside one Prisma `$transaction`, reading the affected row(s) with `SELECT ... FOR UPDATE` before mutating, guaranteeing atomicity and correct rollback on any failure (§21).

H. Concurrency Strategy

Pessimistic row-level locking on the specific `inventory/internal_transfers/work_orders` row involved in each operation — the simplest mechanism that provably prevents the case study's core race conditions (double reservation, double dispatch, double receive) without client-side retry complexity (§22).

I. Main Business Flows

Inventory → Work Order → Stock Check → (Internal Transfer if short) → Customer Reservation — implemented end-to-end as described in §33, with every state transition validated server-side against the state machines in §23.

J. Technology Stack

React/TypeScript + Express/TypeScript + PostgreSQL/Prisma, fully containerizable via Docker Compose, no provider-specific dependencies (§30).

K. Deployment Considerations

Runs identically via `docker-compose up` locally or on any Node-capable host with a reachable Postgres instance; all environment-specific values come from `.env` (§29);

`NODE_ENV=production` disables Swagger UI and verbose error output while keeping the same codebase (no environment-specific branching logic beyond configuration).

Authentication Flow (reference)

See §4.2 sequence diagram.

Reservation Flow (reference)

See §22.2 worked example and §33 Flow 5.

Transfer State Flow (reference)

See §23.2 state diagram.

Frontend/Backend Request Flow (reference)

See §33 flowchart.